数据结构与算法

期中考试专题可视化讲义

线性表 · 栈与队列 · 二叉树 · 哈夫曼树

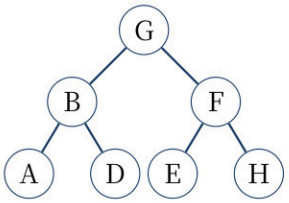
由《某高校　2025—2026 学年第一学期·期中试卷（A）》梳理而成



链表



栈



二叉树

所属课程　《数据结构与算法》（C++ 描述）

适用读者　第一次系统学习本专题、前置基础不牢的本科生

内容　原卷精讲整理 · 答案核验与纠错 · 6 章知识图解 · 38 道分层练习

编译　XeLaTeX，矢量图，A4 打印友好

错”一节。

目　录

阅读说明与本专题定位 3

学科小史：这些结构是怎样出现的? 4

从课本到现实：这些结构到底有什么用? 4

原始资料重新编码：某高校　期中试卷（A） 5

答案独立核验与原图纠错 7

1　线性结构与顺序存储 9

2　链表：用指针把元素“串”起来 13

3　栈：后进先出 18

4　队列：先进先出 22

5　树与二叉树 26

6　哈夫曼树与前缀编码 32

7　知识总览与题目对照 35

8　分层练习（共 38 题） 36

一、基础题（25 题） 36

二、提高题（10 题） 41

三、跨学科题（3 题） 43

9　答案速查 45

10 详解与分析 45

基础题详解 45

提高题详解 51

跨学科题详解 54

11 附录：总结与参考 55

阅读说明与本专题定位

这份讲义是什么

原始资料是一份《数据结构与算法》期中试卷（含同学手写答案）的 5 页扫描件。本讲义没有把试卷“原样排版”，而是做了三件事：把试卷**重新编码**成清晰文字；把答案**逐题独立核验**（发现了需要纠正的地方）；再以试卷覆盖的知识为边界，把线性表、链表、栈、队列、二叉树、哈夫曼树**从“为什么需要它”讲起**，配图、推导、例子、易错点齐全，最后给出 38 道分层练习与带图解析。

颜色与标记

外观	含义
深蓝标题 + 浅蓝灰底	核心知识：定义、定理、公式
暖米白底	例题、生活中的它、学科小史、科普
极浅红底	易错点、原图纠错
极浅绿底	方法总结、学习自检
浅灰蓝 + 左侧线	相似知识对比（含 ✓ △ × 迁移判断）、知识联系

来源标记：[原图识别] 能从原图可靠识别；[教学补全] 原图省略、由本讲义依据标准教材补全；[原图纠错] 原图（含手写答案）有误并已纠正。迁移判断符号：✓ 可直接迁移，△ 加条件成立，× 不成立。

本专题学什么

试卷考的是“**数据怎样组织，才能让常见操作又快又省**”这一条主线上的 6 件事：

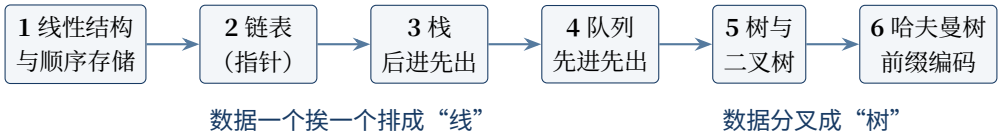


图 0-1　本讲义的知识路线：前四章研究“线”，后两章研究“树”。

前置知识检查（不会也不用慌，正文里都会补）

- C/C++：数组下标、结构体、指针 `p->next`、模板 `template<class T>`；
- 数学：指数与对数（2^k、$\lfloor \log_2 n \rfloor$）、数学归纳法、求和 $\sum$；
- 读代码：能看懂 `while` 循环与指针赋值的先后次序。

它在课程体系中的位置

前面：C++ 程序设计、离散数学（集合、关系、归纳法）。 **后面**：排序与查找（要用到数组、链表、树）、图论算法（树的推广）、操作系统（队列调度、栈帧）、编译原理（栈、表达式树）、信息论与通信（哈夫曼编码、熵）。

学科小史：这些结构是怎样出现的？

问题先于答案

早期计算机的内存只是一长条编号的存储单元。程序员很快遇到三个麻烦：**函数/子程序调用完怎么回到原处**（要记住“最近一次”的位置）；**数据要不断插入删除，不能每次整体搬家**；**怎样用尽量少的二进制位表示信息**。下面这些结构，就是在回答它们。

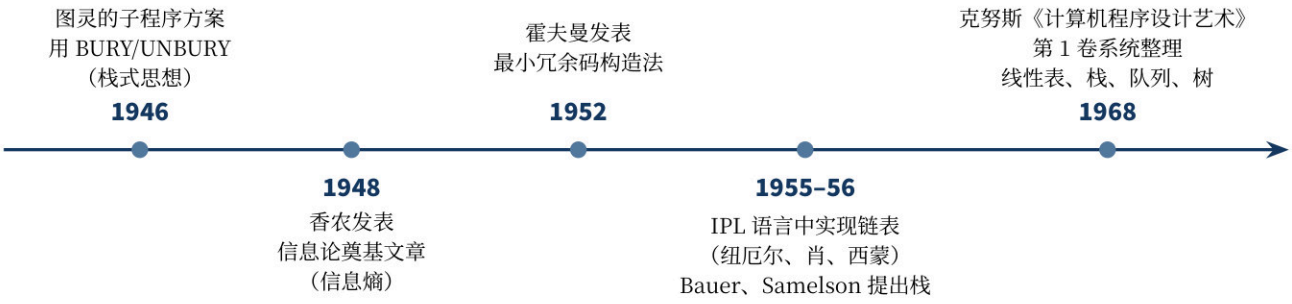


图 0-2　关键节点时间轴（事实来源见末尾参考资料；“最早提出者”存在多人独立工作，仅取保守说法）。

三句话读懂这条时间线

栈：1945—46 年图灵为 ACE 机设计的子程序调用就已用“埋存/取出”的栈式做法；1955 年 Samelson 与 Bauer 把它明确为“Operationskeller”（栈）并于 1957 年申请德国专利，Hamblin 也在 1954 年前后独立提出类似想法。 **链表**：1955—56 年，纽厄尔、肖、西蒙在 RAND 公司为 IPL 语言设计了第一个计算机科学意义上的链表实现，用于“逻辑理论家”程序。**哈夫曼编码**：1951 年霍夫曼还是 MIT 博士生，为交一份“寻找最优二进制码”的学期报告想出了自底向上合并的树，1952 年发表；它在“逐符号编码”的前缀码中是最优的。

从课本到现实：这些结构到底有什么用？

四个你每天都在用的例子

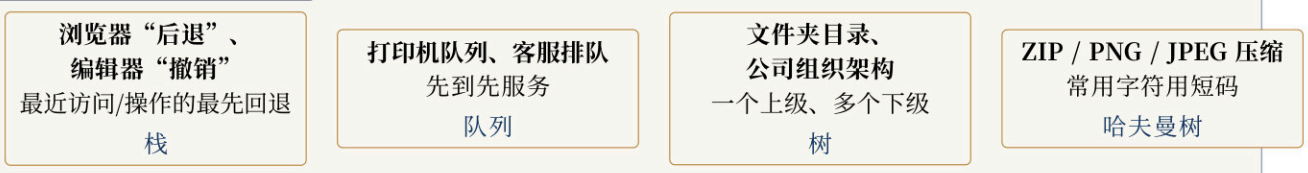


图 0-3　四类现实系统对应的数据结构。

压缩软件不是魔法：常见的 DEFLATE（ZIP、PNG 所用）与 JPEG 的熵编码阶段都用到哈夫曼编码，思想正是第 6 章要学的“出现越频繁，码越短”。**操作系统**用队列安排进程排队，用栈保存函数调用现场；**编译器**用栈检查括号配对、用表达式树生成计算顺序。课堂上的小题，其实是这些真实系统的“微缩模型”。

原始资料重新编码：某高校　期中试卷（A）

试卷信息

某高校　2025～2026 学年第 1 学期《数据结构与算法》课程期中试卷（A），页眉有手写“限时 90 分钟”。共四大题：一、单项选择（10 题×2 分=20 分）；二、填空（每空 2 分，共 20 分，7 小题共 10 个空）；三、简答（6 小题，共 40 分）；**四**、编程（2 小题，共 20 分）。满分 100 分。

识别说明：照片带水印、倾斜、局部模糊。下文已逐字核对；**无法可靠辨认或原图缺失的地方都明确写出，不做猜测**。

一、单项选择题（每题 2 分）

1. 下列数据结构中，哪一个不是线性数据结构？ A. 队列　B. 栈　C. 顺序表　D. 树
2. 在栈中，元素出栈的操作是在栈的哪个端进行的？ A. 栈顶　B. 栈底　C. 任意位置　D. 栈顶或栈底
3. 设计一个判断表达式中左右括号是否配对的算法，采用（　）数据结构最佳。 A. 顺序表　B. 栈　C. 队列　D. 矩阵
4. **（原题不完整）**原卷该题被手写划掉，旁注“题不完整，类似于”，并补写：“若栈的输入序列为 $P_1, P_2, \ldots, P_n$，输出序列为 $1, 2, 3, \ldots, n$，若 $P_3 = 3$，则 P_1 的值（　）”，选项 A“可能是 2”，其余选项文字模糊（辨认为“不可能是……、一定是 1、一定是 2”一类）。**被划掉的原题干无法可靠辨认**。
5. 循环队列存储在数组 $A[0..m]$ 中，则入队时队尾指针 `rear` 的操作为（　） A. `rear=rear+1`　B. `rear=(rear+1) mod (m-1)`　C. `rear=(rear+1) mod m`　D. `rear=(rear+1) mod (m+1)`
6. 在 C/C++ 语言中，数组在内存中是按行优先顺序存储的。若定义一维数组 `a[50]` 和二维数组 `b[5][10]`，且 `&a[0]=&b[0][0]`，则 `a[18]` 的地址和（　）的地址相同。 A. `b[1][7]`　B. `b[1][8]`　C. `b[8][1]`　D. `b[7][1]`
7. 深度为 k 的完全二叉树，最少有（　）个结点？ A. $2^{k-1}+1$　B. 2^{k-1}　C. $2^{k-2}-1$　D. 2^{k-2}
8. 假设我们将一个表达式表示为一棵二叉树：如 `a+b*c` 对应的树，根为 `+`，其左子树是一个叶子结点 `a`，右子树的根为 `*`，左右子结点分别为 `b` 和 `c`。显然每个内部结点代表一个运算。假设一个并行 CPU 有很多个 ALU，可以同时执行计算任务，且完成每个运算需要的时间都是一个时间单位。那么完成该表达式计算的最短时间是（　） A. 树的高度−1　B. 树的宽度　C. 树的内部结点个数　D. 树的分支数
9. 假设有一棵二叉树的结点前序排列是 1,2,3,4,5。下面的（　）序列不可能是这棵二叉树的中序排列。 A. 3,2,1,4,5　B. 2,3,1,4,5　C. 2,1,4,5,3　D. 2,1,5,3,4
10. 下述编码中（　）不可能是前缀编码。 A. (00,01,10,11)　B. (0,1,00,11)　C. (0,10,110,111)　D. (1,01,000,001)

二、填空题（每空 2 分，共 20 分）

1. 在栈中，元素的进出顺序是 ____，在队列中，元素的进出顺序是 ____。
2. 在链表中，每个结点包含 ____ 和 ____ 两部分。
3. 在线性表的顺序存储中第一个元素的存储地址是 100，每个元素的长度是 2，则第 5 个元素的存储地址是 ____。
4. 完全二叉树按照层次顺序，自顶向下，同层从左到右顺序从 0 开始编号时，编号为 i 的结点的左子结点的编号为 ____。
5. 设栈 S 和队列 Q 的初始状态皆为空，元素 a, b, c, d, e, f 依次通过一个栈，一个元素出栈后即进入队列 Q，若 6 个元素的出队列的顺序是 c, e, d, f, b, a，则栈 S 至少应该容纳 ____ 个元素。（元

素表与出队顺序的个别字母为照片轻微模糊处，按此识别，经上下文自洽。）

6. 设单链表中指针 p 指向结点 A，若要删除 A 的后继结点（假设 A 存在后继结点），则需修改指针的操作为（写出 C++ 语言代码）_____。

7. 以下算法实现了循环链表的逆转。

```
template<class Type>
void CList<Type>::Inverse(){
  CListNode<Type> *p, *q, *r;
  if (first->link == first) return;
  p = first->link; q = p->link;
  p->link = first;
  while ( ____(1)____ ) {
    r = q->link; q->link = p;
    p = q; q = r;
  };
  ____(2)____
}
```

请在算法的横线上填入适当的语句，每处只填一个语句或一个表达式。

三、简答题（6 小题，共 40 分）

1. **原图缺失**：提供的扫描件第 2 页右栏从第 2 小题开始，第 1 小题题干未出现，同学答案也只写“略”。由总分 40 与其余 5 题分值（$6+6+6+8+8=34$）推测第 1 小题为 6 分——**这只是推测，不属于原图识别结果**。

2. 设计一个算法，判断一个给定的链表是否为回文结构。回文是指正读反读均相同的字符序列，如“abba”和“abdba”均是回文，但“good”不是回文。简要描述算法的思路即可，另外简要说明其时间复杂度。（6 分：写出算法思路 3 分，写对时间复杂度 3 分）

3. 有 5 个元素，其进栈次序为 A、B、C、D、E，在各种可能的出栈次序中，以元素 C、D 最先出栈（即 C 第一个出，D 第二个出栈）的次序有哪些？（6 分，写出一条给 2 分）

4. 请把下面的森林转化为二叉树并画出二叉树结构。（6 分）
森林含 3 棵树：T_1：根 A，A 的三个孩子依次为 B、C、D，B 有一个孩子 E；T_2：根 F，F 有一个孩子 G；T_3：只有一个结点 H。

5. 已知一棵二叉树的先序遍历（Pre-order）和中序遍历（In-order）序列如下，请画出这棵二叉树的唯一结构，并给出后序遍历序列。（8 分，画出图 6 分，写出后序遍历序列 2 分）
先序：G, B, A, D, C, F, E, H　中序：A, B, C, D, G, E, F, H

6. 对给定的一组权值 $W=(5,2,9,11,8,3,7)$，试构造相应的哈夫曼树，并计算其带权路径长度。（8 分，构造出图给 6 分，算出路径长度给 2 分）

四、编程题（2 小题，共 20 分，答案填写在答题卡上）

1. 已知一个带头结点的单链表按照值递增排列，基于 C++ 代码构造一个算法，删除链表中值为 x 的结点。若结点不存在，则函数返回 0，若删除成功则返回 1。（10 分）

```
int DeleteX(LinkNode *&h, int x);   // 函数原型
typedef struct __LinkNode {          // 结点定义
  int data;
  __LinkNode *next;
} LinkNode;
```

2. 一棵具有 n 个结点的二叉树，采用二叉链式存储结构，基于 C++ 代码编写算法对该二叉树进行层次遍历（利用队列实现 LinkQueue）。（10 分）

```
LevelOrder(void (*visit)(const ElemType &))   // 函数原型
typedef struct BinTreeNode {
  ElemType data;                        // 数据成分
  BinTreeNode<ElemType> *leftChild;     // 左孩子指针成分
  BinTreeNode<ElemType> *rightChild;    // 右孩子指针成分
};
```

答案独立核验与原图纠错

先说结论

原答案页写着“答案为本人自己整理，可能有错，如果感觉不对请问 AI”。我把全部可判定的题目**独立重新求解，并用程序做了交叉验算**（出栈序列合法性、哈夫曼合并、由遍历序列建树、循环链表逆转模拟、前缀码判定）。结果：**选择题 10 题全部正确；填空第 5 题与第 7 题两处有误，第 6 题写法需规范；简答与编程的思路基本正确；简答第 1 题题干缺失，简答第 6 题原答案为“略”，已补全。**

题号	同学答案	本讲义核验结果	结论
选 1–3	D, A, B	树是非线性；出栈在栈顶；括号匹配用栈	✓
选 4	A（题不完整）	在补全题意下 P_1 可取 1、2、4…，故“可能是 2”成立	✓
选 5–6	D, B	$A[0..m]$ 共 $m+1$ 个单元；$18 = 1 \times 10 + 8$	✓
选 7–8	B, A	最少结点数 2^{k-1}；时间 = 内部结点层数 = 高度−1	✓
选 9–10	D, B	D 违反栈序；B 中 0 是 00 的前缀	✓
填 1–4	LIFO/FIFO；数据域/指针域；108；$2i+1$	均正确	✓
填 5	**3**	栈最大深度应为 **4**（见下）	×
填 6	`p->next=p->next->next;`	方向对，但丢失被删结点（内存泄漏）；规范写法见下	△
填 7	`q!=NULL; q->next=p;`	应为 `q!=first; first->link=p;`（见下）	×
简 1	略	原图缺失，无法核验	—
简 2–5	找中点、反转后半、比较，$O(n)$；CDEBA 等 3 条；森林图；后序 ACDBEHFG	均已核对	✓
简 6	略	补全：WPL = 120	—
编 1–2	参考答案	思路正确；`DeleteX` 参考答案漏了 `p=h` 初始化	△

纠错 1：填空第 5 题 [原图纠错]

原始写法 → 问题 → 规范写法

原始写法：答案“3”。

问题：同学可能只数到 a, b, c 同时在栈中的 3 个，漏看了后面 a, b, d, e 同时在栈中的时刻。

规范写法：栈至少应容纳 **4** 个元素。

为什么这样改：逐步模拟 push a, b, c; pop c; push d, e; pop e, d; push f; pop f, b, a。c 出栈后栈中还剩 a, b；再压入 d, e 时栈中是 a, b, d, e，共 4 个（图 S-1 峰值）。因为队列只是把栈的出栈顺序原样保存，所以队列出队序列 = 栈的出栈序列。

是否影响后续：不影响其他题；但它说明“栈容量”要看**整个操作过程的最大深度**，第 3 章会系统讲。

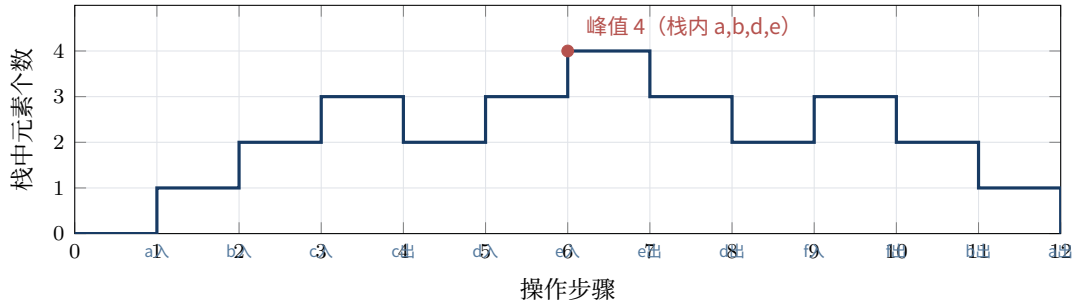


图 S-1　填空 5：栈深度随操作的变化。横轴每格是一次入栈或出栈；峰值出现在第 6 步。

纠错 2：填空第 7 题（循环链表逆转）[原图纠错]

原始写法 → 问题 → 规范写法

原始写法：(1) `q != NULL`（或 `q`）；(2) `q->next = p;`。
问题：① 这是**循环**链表，最后一个结点的指针指回 `first`，`q` 永远不会为 `NULL`，循环不会停；② 本题结点的指针域名叫 `link`，不是 `next`；③ 循环结束时 `q` 已回到 `first`，`q->link=p` 等于改了头结点自己的指针，且位置、含义都不对。
规范写法：(1) `q != first`；(2) `first->link = p;`。
为什么这样改：循环终止条件是“`q` 绕了一圈回到头结点”；退出时 `p` 指向原来的最后一个结点，它现在应成为新的第一个结点，所以让头结点的 `link` 指向 `p`。已用 0～5 个结点的链表程序模拟，结果均为逆序（详见 2.5 节图 2-6）。
是否影响后续：2.5 节、练习第 21–25 题的逆置算法都以此为例。

纠错 3：填空第 6 题（删除后继）[原图纠错] △

原始写法 → 规范写法

原始写法：`p->next = p->next->next;` **问题**：指针修改方向正确，但被删结点没有被释放，会造成内存泄漏；另外本题链表的指针域名在不同小题中分别叫 `link`（小题 7）与 `next`（编程题 1），按所用结点定义书写即可。
规范写法：`q = p->next; p->next = q->next; delete q;`（`q` 为 `LinkNode*`）。**只考“修改指针”时，第一句加第二句即满分；`delete` 属于规范习惯。**

纠错 4：编程题 1 的参考答案 [原图纠错] △

参考答案中循环 `while(p->next!=nullptr && p->next->data<x)` 使用了 `p`，但**没有写 `LinkNode *p = h;`**（图像上未见该句）。完整可运行版本见 2.4 节。另外题目没有说明链表中是否有重复值；若可能重复，应一直删除到值大于 `x` 为止（本讲义同时给出两种版本）。

其他需要说明的地方

- **选择 4**：被划掉的原题干无法辨认，答案 A 在“补全后的题意”下成立，但**不能证明原题就是这道题**。
- **选择 8**：答案“树的高度−1”隐含约定**高度按层数计（叶子所在层也计 1）**。若按边数计高度，则答案应是“树的高度”。选项只有“高度−1”，所以采用层数约定；这一点在第 5 章 5.6 节展开。
- **简答 1 / 6**：简答 1 题干缺失，无法编写；简答 6 原答案“略”，本讲义补全于第 6 章。

第 1 章　线性结构与顺序存储

本章回答三个问题 **(1)** 数据之间的“关系”有哪几种，为什么栈、队列、表是“线性”的而树不是？ **(2)** 把线性表放进数组时，第 i 个元素的地址怎样一步算出？ **(3)** 二维数组在内存里怎样“拉直”？——对应试卷：选择 1、6，填空 3。

1.1 为什么先要分“线性”与“非线性”

同样是一组数据，可以排成一排、可以分叉成树、也可以互相连成网。**关系不同，能做的操作和算法就不同**：排成一排的数据，“第一个、下一个、最后一个”有明确含义；分叉的数据就要问“哪个是父亲、哪个是孩子”。

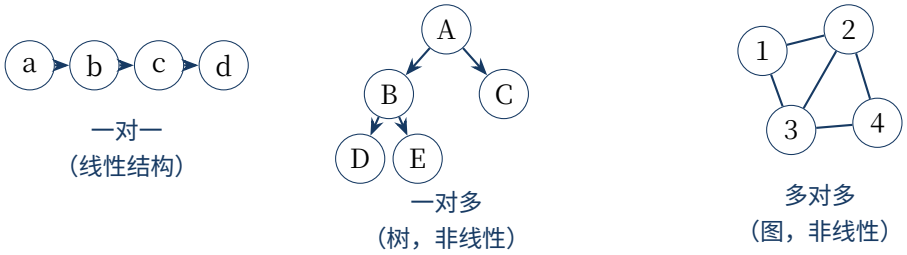


图 1-1　数据元素之间三种基本关系。箭头由“前一个”指向“后一个”。

线性结构（线性表）

若一个非空数据结构满足：**有且仅有一个**“第一个”元素（没有前驱）；**有且仅有一个**“最后一个”元素（没有后继）；其余每个元素都**恰有一个直接前驱和一个直接后继**——则称它为线性结构。形式上，线性表是 n $(n \ge 0)$ 个同类型元素的有限序列

$$L = (a_1, a_2, \ldots, a_n),$$

a_i 的前驱是 a_{i-1}，后继是 a_{i+1}；$n = 0$ 称空表。

	线性结构	非线性结构
典型代表	顺序表、链表、**栈**、**队列**、字符串	**树**、二叉树、图
元素间关系	一对一	一对多（树）、多对多（图）
为什么	每个元素最多一个前驱、一个后继	一个元素可以有多个孩子或多个邻居

对应试卷：选择题 1 [原图识别]

“下列数据结构中，哪一个不是线性数据结构？队列 / 栈 / 顺序表 / 树。” **思路**：逐个问“每个元素是不是最多一个前驱、一个后继”。队列：排队，前面一个后面一个，是；栈：叠起来，上一个下一个，是；顺序表：是；树：一个结点有多个孩子，不是。答案 **D（树）**。

易错点

把“线性结构”和“顺序存储”混为一谈。“线性”说的是元素之间的逻辑关系；“顺序”说的是存放在内存里的方式。链表也是线性结构，只是不用连续内存。同样，树也可以存放在数组里（第 5 章），但它仍然是非线性结构。

1.2 顺序存储：第 i 个元素的地址怎样一步算出

顺序存储就是把线性表的元素**依次存放在地址连续的内存单元**里。设每个元素占 c 个存储单元（字节），第一个元素地址为 $\mathrm{Loc}(a_1)$。

顺序表的地址公式

$$\boxed{\mathrm{Loc}(a_i) = \mathrm{Loc}(a_1) + (i-1)\times c \quad (1 \le i \le n)}$$

若元素按 C/C++ 习惯从下标 0 开始编号（数组 `a[0..n-1]`），则

$$\mathrm{Loc}(\mathrm{a}[i]) = \mathrm{Loc}(\mathrm{a}[0]) + i \times c.$$

符号：Loc 表示“存储地址”；c 为每个元素的字节数；i 为元素序号（从 1 数起）。**条件**：元素类型相同（因而大小同为 c），且存放在连续内存。

为什么是 $(i-1)\times c$？（教学补全）

要证：第 i 个元素地址 $=$ $\mathrm{Loc}(a_1) + (i-1)c$。 **思路**：数一数“在它前面有几个元素，各占多少”。

证明（对 i 归纳）：$i = 1$ 时，前面没有元素，$\mathrm{Loc}(a_1) = \mathrm{Loc}(a_1) + 0\cdot c$，成立。假设对 i 成立；因为元素连续存放且大小都为 c，a_{i+1} 紧接在 a_i 之后，故

$$\mathrm{Loc}(a_{i+1}) = \mathrm{Loc}(a_i) + c = \mathrm{Loc}(a_1) + (i-1)c + c = \mathrm{Loc}(a_1) + i\,c,$$

即对 $i+1$ 也成立。 □

直观：到第 i 个元素家门口，要“跳过”它前面的 $i-1$ 户邻居，每户占 c 米。 **推论**：下标从 0 开始时偏移量正好是 $i\cdot c$，少一个“-1”，这就是 C/C++ 数组下标从 0 开始的原因之一。

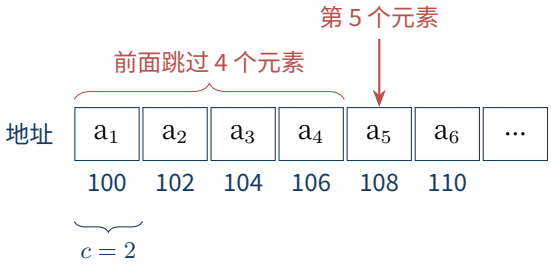


图 1-2　首地址 100、每个元素 2 字节的顺序表。地址每格增加 $c = 2$；第 5 个元素前面有 4 个元素，所以地址 $= 100 + 4 \times 2 = 108$。

对应试卷：填空题 3 [原图识别]

已知首元素地址 100，每元素长 2。**求**第 5 个元素地址。 **第一步**：套公式 $i = 5$，$c = 2$：$100 + (5-1)\times 2$。 **第二步**：$= 100 + 8 = \mathbf{108}$。 **检查**：从 100 往后数 5 个单元的起点：100，102，104，106，108，第 5 个正是 108。 **本题想说明**：是 $i-1$ 而不是 i；如果误算成 $100 + 5\times 2 = 110$，就是第 6 个元素的地址。

顺序表的特点（教学补全） 由公式可见，**只要知道下标，一次乘加就能算出地址**，所以按下标读写任意元素都是 $O(1)$（“随机访问”）。代价是：在中间插入或删除，要把后面的元素整体移动，平均移动约 $n/2$ 个，为 $O(n)$。

1.3 二维数组：怎样把“表格”拉成“一条线”

内存只有一维地址，二维数组 `b[m][n]`（m 行 n 列）要“拉直”存放。C/C++ 采用**行优先（行主序）**：先放第 0 行的 n 个元素，再放第 1 行，……。

行优先地址公式

$$\boxed{\mathrm{Loc}(\mathtt{b}[i][j]) = \mathrm{Loc}(\mathtt{b}[0][0]) + (i \times n + j) \times c} \qquad 0 \le i < m,\ 0 \le j < n.$$

线性化后的“一维下标” $k = i \cdot n + j$；反过来，$i = \lfloor k/n \rfloor$（整除），$j = k \bmod n$。

推导（教学补全）

到 `b[i][j]` 之前，先放完了第 $0, 1, \ldots, i-1$ 共 i 个整行，每行 n 个，共 $i \cdot n$ 个元素；再在第 i 行里放了 $b[i][0..j-1]$ 共 j 个。所以它前面共有 $i \cdot n + j$ 个元素，套用顺序表公式（下标从 0 起）即得。 □

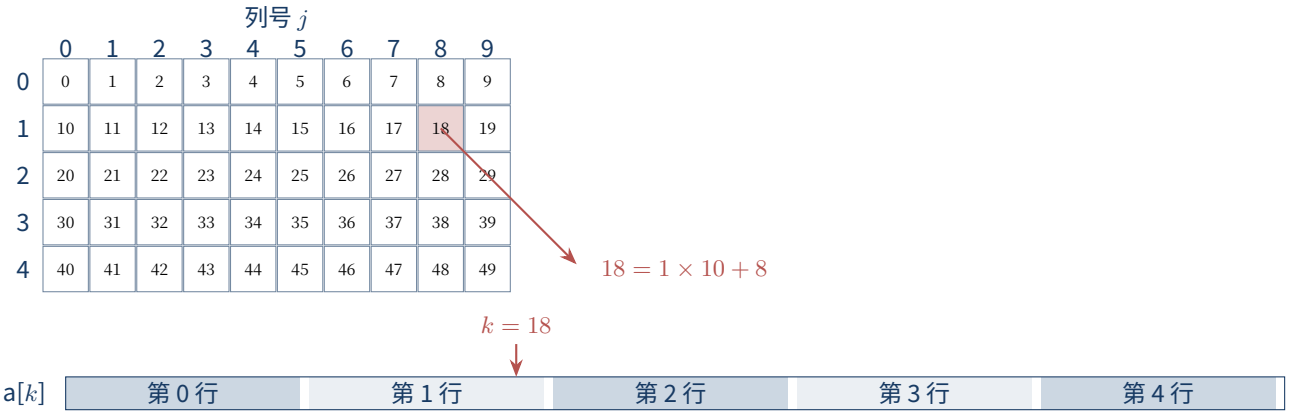


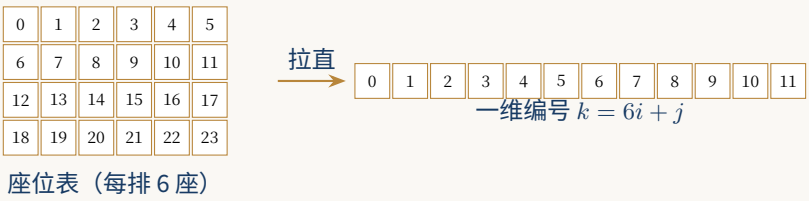
图 1-3　上：5×10 数组，格内数字是行优先的一维下标 $k = 10i + j$。下：拉直后的一维数组，每 10 格一行。红色的 $k = 18$ 对应 $i = 1,\ j = 8$。

对应试卷：选择题 6 [原图识别]

题意：`a[50]` 与 `b[5][10]` 首地址相同（正好 $5 \times 10 = 50$ 个元素），问 `a[18]` 与哪个元素地址相同。 **思路**：两个数组占的是同一段内存、同一顺序，所以“同地址”就是“同一维下标 $k = 18$”。 **计算**：$i = \lfloor 18/10 \rfloor = 1$，$j = 18 \bmod 10 = 8$，即 `b[1][8]`。 **判断各选项**：A `b[1][7]`→ $k = 17$；**B `b[1][8]`**→ $k = 18$；C `b[8][1]`、D `b[7][1]` 的行号 ≥ 5，越界（只有 0～4 行）。答案 B。 **易错**：看到 18 就写 `b[1][18]`（列号越界），或把 `b[8][1]` 当成 $8 \times 1 + 1$ 的“列优先”想法。

生活中的它 · 电影院座位号

生活场景：电影票“第 3 排 4 座”，系统内部常把全场座位排成一串编号，方便存储和统计。
对应关系：排号 ↔ 行号 i；座号 ↔ 列号 j；每排座位数 ↔ 列数 n；内部座位编号 ↔ 一维下标 $k = i \cdot n + j$。
小估算：每排 20 座，座位从 0 编号，第 3 排（从 0 数起）第 4 座：$k = 3 \times 20 + 4 = 64$。反过来，编号 105 的座位在 $i = \lfloor 105/20 \rfloor = 5$ 排、$j = 105 \bmod 20 = 5$ 座。
边界：真实影厅常常每排座位数不同（前排少、后排多），这时就不能用统一的 n，要像“不规则数组”那样为每一行单独记录起点。



对照：行优先拉直 ↔ 矩阵的向量化（推广型 / 同名异义型）

一句话定位：把二维表“拉”成一条线，C/C++ 的**行优先**与线性代数、MATLAB 里常用的“**列向量化**” $\mathrm{vec}(A)$（按列堆叠）是同一思想的两种约定。

维度	C/C++ 二维数组	线性代数 / MATLAB 矩阵
下标起点	从 **0** 开始：`b[0][0]`	从 **1** 开始：a_{11}
拉直顺序	行优先：0 行全部，再 1 行	$\mathrm{vec}(A)$ 与 MATLAB 的 `A(:)`：列优先
一维下标公式	$k = i \cdot n + j$	$k = (j-1) \cdot m + i$（1 起，m 行）
目的	把二维地址映射到一维内存	把矩阵写成向量，便于做线性运算

相同：都是**双射**（一一对应）：二维位置 ↔ 一维位置，且可逆。 **不同及根源**：顺序由语言规范约定，没有“对错”，只有“约定”；所以同一份内存数据，用行优先与列优先**读出来的矩阵互为转置**。

旧知识中的结论	迁移	理由或反例
“一维下标 = 行号×列数 + 列号”	×	列优先时应是 列号×行数 + 行号；下标从 1 起时还要减 1
“知道总元素个数就能由一维下标还原行列”	△	必须同时知道列数 n（行优先）或行数 m（列优先）
“拉直后与原表一一对应、可逆”	✓	两种约定都是双射

同一个小算例：3×2 矩阵 $\begin{pmatrix}1 & 2\\ 3 & 4\\ 5 & 6\end{pmatrix}$，行优先拉直 $= (1,2,3,4,5,6)$，列优先拉直 $= (1,3,5,2,4,6)$。

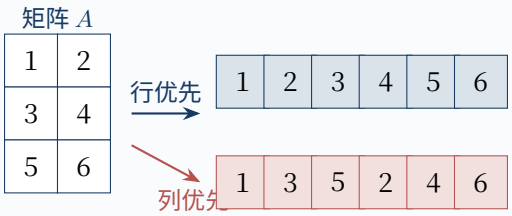


图 1-4　同一个 3×2 矩阵的两种拉直方式：左矩阵；右上行优先（C/C++），右下列优先（MATLAB、Fortran）。

学完这里，你应该能回答

1. 线性结构的“前驱后继”条件是什么？为什么树不是？
2. 顺序表第 i 个元素地址公式是什么？为什么有 $i-1$？
3. `b[i][j]` 在行优先下是第几个元素？反过来 $k = 18$ 对应哪一行哪一列（$n = 10$）？
4. “线性结构”和“顺序存储”有什么区别？链表属于哪一类？
5. 行优先与列优先读出的矩阵有什么关系？

到这里，我们已经解决了什么？下一步还缺什么？

我们知道了“线性”是什么，也看到**顺序存储用一个公式就能随机访问**，但**插入删除要大量移动元素**。下一章用**指针把元素“串”起来**——不要求连续内存，插入删除只需改几个指针。后续联系：数组是排序、查找、堆、散列表的底层；二维数组的行主序在图像处理、矩阵计算中随处可见。

第 2 章　链表：用指针把元素“串”起来

本章回答 为什么顺序表“插入删除慢”？链表怎样做到“只改几个指针”？有序链表删除指定值、循环链表逆转、判断回文——这些是怎样一步步想出来的？（对应试卷：填空 2、6、7，简答 2，编程 1。）

2.1 为什么需要链表

第 1 章末尾我们看到：顺序表在中间**插入**一个元素，要把它后面的所有元素向后移动一格；**删除**则要向前移动。元素越多，搬得越多。能不能让元素“**各住各的**，只记录下一个住在哪里”？这就是链表。

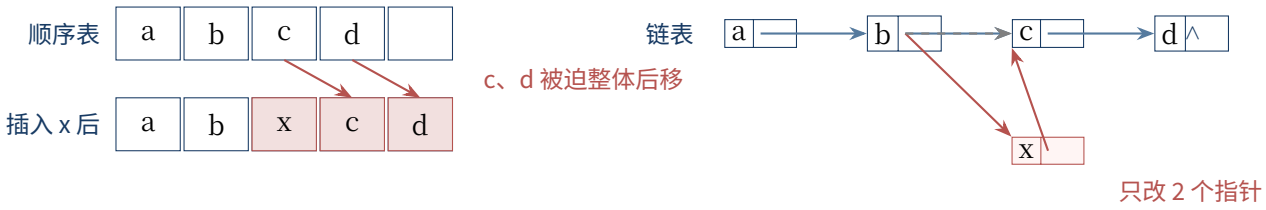


图 2-1　插入 x：顺序表要移动其后所有元素（红）；链表只改两根指针（红），其余结点原地不动。

2.2 结点与单链表

结点、单链表

结点由两部分组成：**数据域**（存放元素本身）和**指针域**（存放**下一个结点的地址**）。**单链表**就是由这样的结点首尾相接形成的线性表：每个结点的指针域指向它的直接后继，**最后一个结点的指针域为空**（nullptr，图中记 ∧）。 常见约定：**头指针**指向链表第一个结点；**带头结点**的链表在第一个数据结点前多放一个“哨兵”结点（数据域不用），使头指针永远指向它。

```
typedef struct __LinkNode {   // 结点定义（与试卷编程题 1 相同）
  int data;                   // 数据域
  __LinkNode *next;           // 指针域：指向后继结点
} LinkNode;
```

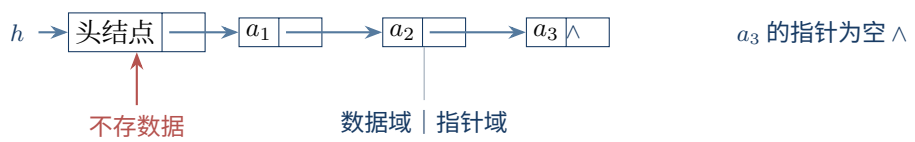


图 2-2　带头结点的单链表。每个方框左半是数据域，右半是指针域；箭头表示“存放着对方的地址”。

2.3 删除后继结点（填空 6）

删除 p 的后继结点 q

设 p 指向某结点，要删除它后面的那个结点 q：

```
LinkNode *q = p->next;     // 1. 先把被删结点“记住”
p->next = q->next;         // 2. 让 p 直接指向 q 的后继（跳过 ）q
delete q;                  // 3. 释放 q 占用的内存
```

只考“修改指针”时，核心就是第 2 句，等价于 p->next = p->next->next;。

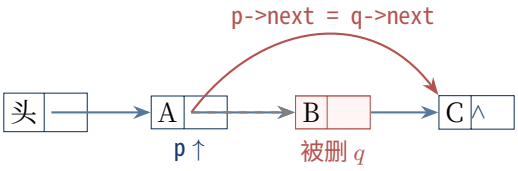


图 2-3　删除 A 的后继 B：红色弧线是新的指针（A 直接指向 C），灰色虚线是被切断的旧指针。

易错点：顺序一乱就找不到 q 与 插入时的先后

(1) 若先写 delete p->next; 再 p->next = p->next->next;，第二句已在使用被释放的内存，结果不可预测。 (2) **插入**（在 p 之后插入 s）必须先让 s 接上后面，再让 p 指向 s：

```
s->next = p->next;     // 先：s 指向原来的后继
p->next = s;           // 后：p 指向 s
```

若颠倒，p->next=s 后 p->next 已是 s，再执行 s->next=p->next 就成了 s->next=s，**自己指自己**，后面整条链丢失。

2.4 有序表中删除值为 x 的结点（编程题 1）

问题：带头结点的单链表按值**递增**，删除值为 x 的结点；不存在返回 0，删除成功返回 1。

想法：删除一个结点必须先找到它的**前驱**（要改前驱的指针）。所以不要让 p 停在目标结点上，而要让它停在“**后继的值** $\geq x$”的那个结点上——因为链表有序，一旦 p->next->data 不再小于 x，后面都更大，就不用再往后找了（这是有序带来的剪枝）。

```
int DeleteX(LinkNode *&h, int x) {
  LinkNode *p = h;                          // p 从头结点出发
  while (p->next != nullptr && p->next->data < x)
    p = p->next;                            // 前进，直到后继 >= x 或到表尾
  if (p->next == nullptr || p->next->data != x)
    return 0;                               // 没有值为 x 的结点
  LinkNode *q = p->next;                    // q 即要删的结点
  p->next = q->next;                        // 跳过 q
  delete q;
  return 1;                                 // 删除成功
}
```

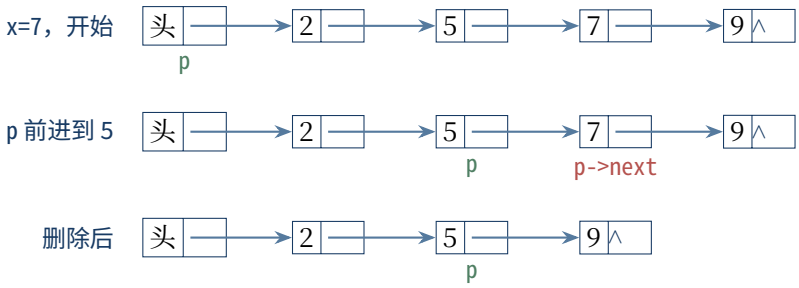


图 2-4　x=7 时的执行过程：p 停在 5（后继 7 不再小于 7），确认 p->next->data==7，然后摘掉。

为什么这样写是对的?

循环不变式：每次判断循环条件时，p 及其之前所有结点的值都 $<$ x。起始时 p 为头结点（没有数据，视为成立）；每次 p 前进到 p->next，而进入循环的条件正是 p->next->data < x，所以不变式保持。循环结束有两种可能：p->next==nullptr（所有结点都 $<$ x，没有 x）；或 p->next->data>=x。由有序性，若表中有 x，它只能是 p->next。**注意 && 的顺序**：先判 p->next!=nullptr，利用短路求值避免对空指针取 ->data。 **时间**：最坏扫描整表，$O(n)$。

教学补全 · 引用参数 *&h 与重复值

本题有头结点，h 本身永远不变，*&h 并非必需；若是**不带头结点**的链表，删除第一个结点要改 h，就必须按引用传递。若表中可能有多个值为 x 的结点，把最后几句改为：

```
  int cnt = 0;
  while (p->next != nullptr && p->next->data == x) {
    LinkNode *q = p->next; p->next = q->next; delete q; cnt++;
  }
  return cnt > 0 ? 1 : 0;
```

2.5 循环链表与逆转（填空 7）

循环链表

把单链表**最后一个结点的指针改为指向头结点**（或第一个结点），整条链首尾相接成环，就是循环链表。好处：从任一结点出发都能走遍整个表；**判断“走完一圈”不能再用 nullptr，要比较是否回到出发点**（本题是 q != first）。

试卷给出的算法（first 为头结点，指针域名 link）：先处理前两个结点，然后让 q 沿原方向前进，每次把 q->link 反过来指向 p，最后让头结点指向新的第一个结点。

```
template<class Type>
void CList<Type>::Inverse(){
  CListNode<Type> *p, *q, *r;
  if (first->link == first) return;          // 空表
  p = first->link; q = p->link;              // p: 已逆转部分的头; q: 待处理
  p->link = first;                           // 原第一个结点变成新的尾，指回头结点
  while ( q != first ) {                     // (1) 还没绕回头结点
    r = q->link;                             // r 先记住后继，防止断链
    q->link = p;                             // 反转 q 的指针
    p = q; q = r;                            // 、pq 同时前进一格
  };
  first->link = p;                           // (2) p 现在是原来的最后一个结点
}
```

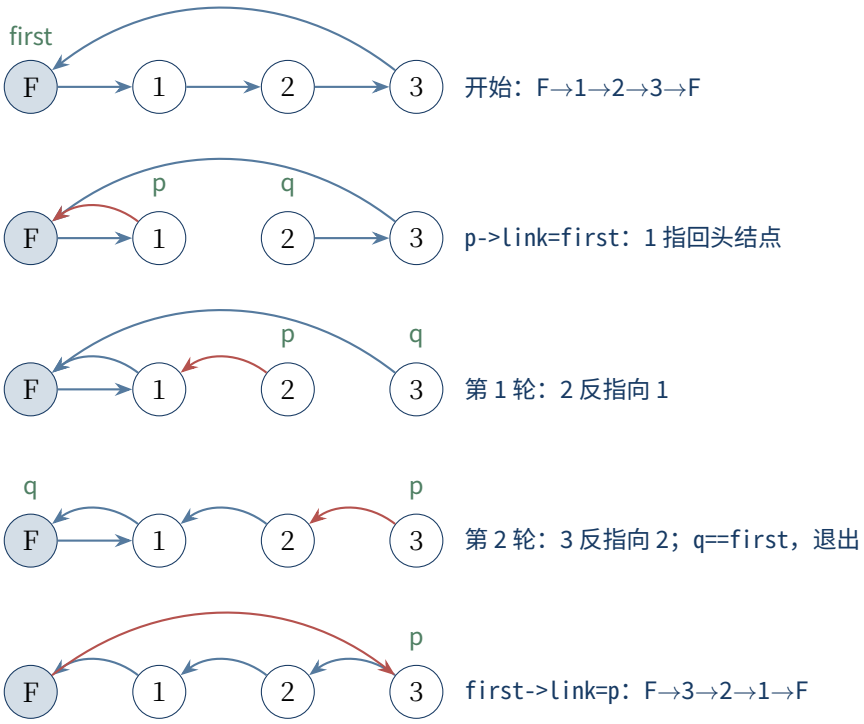


图 2-6　3 个数据结点的逆转过程（F 为头结点）。红色箭头是本步新写的反向指针；绿色字母是此刻 p、q 的位置。

为什么原答案 q!=NULL、q->next=p 是错的?

循环链表中没有指针为空，`q!=NULL` 永远成立，程序死循环；退出条件必须是回到头结点 `q!=first`。另外退出时 q 正好是头结点，应修改的是**头结点的指针** `first->link=p`，字段名也是 `link`。

2.6　判断链表是否回文（简答 2）

问题：判断单链表是否正读反读相同。如 $abba$、$abdba$ 是，$good$ 不是。**单链表只能从前往后走**，没法像数组那样两头向中间比，怎么办?

算法思路（时间 $O(n)$，额外空间 $O(1)$）

(1) 用**快慢指针**找中点：slow 每次走 1 步，fast 每次走 2 步；fast 到表尾时 slow 恰在中点。 **(2)** 把**后半部分**链表就地**反转**（用上一节的“逆置”思路）。 **(3)** 两个指针分别从**表头**和**反转后后半段的头**同时出发逐个比较，全部相等即为回文；奇数长度时中间结点不参与比较。 **(4)**（可选）再反转回来恢复原链表。

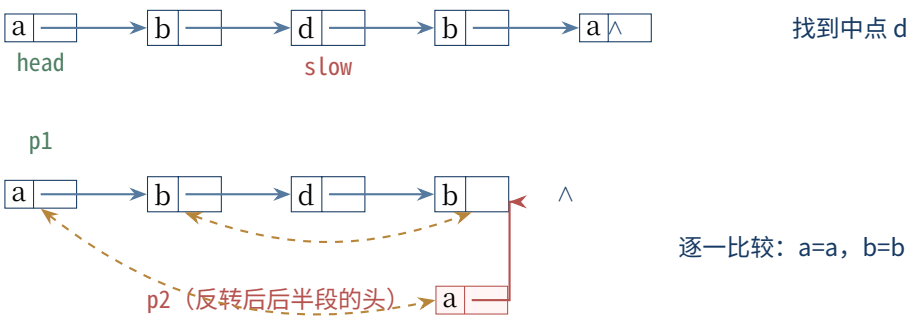


图 2-7　$abdba$ 的判断：找到中点 d，把 d 之后的 b a 反转为 a b，再与前半段 a b 逐个比较（金色双向箭头）；中间的 d 不参与比较。

为什么快慢指针能找到中点？（教学补全）

走 t 步后 slow 走过 t 个结点，fast 走过 $2t$ 个结点。设表长为 n，fast 走到尾（约 $2t \approx n$）时 $t \approx n/2$，即 slow 恰在中间。**时间复杂度**：找中点 $\approx n/2$ 步，反转后半 $\approx n/2$，比较 $\approx n/2$，合计 $\approx 1.5n$，所以是 $O(n)$；只用了几个指针变量，额外空间 $O(1)$。

替代方法（对比）：把全部结点值压入栈，再依次出栈与链表从头比较——也是 $O(n)$ 时间，但要 $O(n)$ 额外空间。

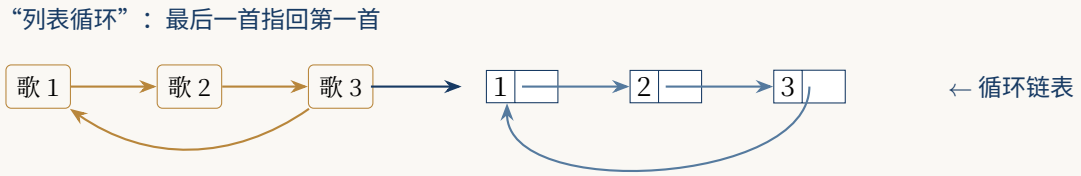
生活中的它 · 播放列表

场景：音乐软件的播放列表，“下一首”由上一首指定。

对应：一首歌 ↔ 结点；“接下来播放谁” ↔ 指针域 next；单曲循环/列表循环 ↔ 循环链表。

小估算：1000 首歌的列表，把一首新歌塞到第 500 首后面。用数组要移动约 500 首；用链表只需改 2 个“下一首”。但要先找到第 500 首，需要沿链走约 500 步——所以链表的优势在于**已知位置之后的插入/删除**。

边界：真实软件常同时保存“上一首”和“下一首”（双向链表），并配合索引加速查找。



2.7 顺序表与链表：怎么选

比较维度	顺序表（数组）	单链表
存储	连续内存，元素紧挨	结点分散，靠指针相连
按序号访问第 i 个	$O(1)$（地址公式）	$O(i)$（从头数起）
已知位置的插入/删除	$O(n)$，要移动元素	$O(1)$，只改指针
额外空间	无（预留容量可能浪费）	每个结点多存一个指针
容量	需事先确定或扩容	随用随分配
典型适用	查得多、变得少	增删多、长度不定

对照：链表的“递归结构” ↔ 数学归纳法（思想重现型）

一句话定位：链表可以递归地定义——“空表，或者一个结点接一个更短的链表”——这与数学归纳法“基础情形 + 归纳步骤”是同一个思想。

维度	链表（程序）	数学归纳法
“基础”	空表（nullptr）	$n = 0$ 或 1 时命题成立
“递推”	链表 = 首结点 + 剩余链表	若 n 成立则 $n+1$ 成立
要写/要证的对象	算法（求和、查找、逆置）	命题 $P(n)$
出错的常见点	漏了空表（p==nullptr）	漏了基础情形

旧知识中的结论	迁移	理由或反例
“归纳可以证明对所有有限链表成立”	✓	链表长度 $n \in \mathbb{N}$，对 n 归纳即可
“对循环链表同样可用归纳”	×	循环链表没有“更短的剩余链表”，递归不会终止，须改用“走回起点”作终止条件
“证明 $P(n) \Rightarrow P(n+1)$ 就够”	△	还要验证基础情形，且空表/单结点要单独检查

同一个小算例：求链表 $(3,5,2)$ 各结点之和。归纳/递归：$\mathrm{sum}(\text{空}) = 0$，$\mathrm{sum}(a{:}R) = a + \mathrm{sum}(R)$，得 $3 + (5 + (2 + 0)) = 10$。迭代：p 从头走一遍，累加 $3 \to 8 \to 10$。结果相同——递推就是循环的数学说明书。

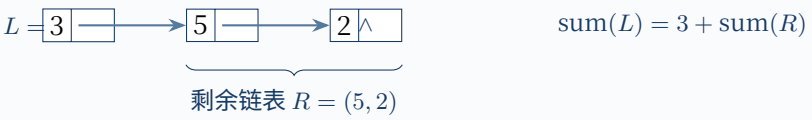


图 2-8　链表 = 首结点 + 剩余链表：“同样结构、规模更小”正是递归与归纳的共同点。

学完这里，你应该能回答

1. 结点由哪两部分组成？头指针与头结点有什么区别？
2. 删除 p 的后继、在 p 之后插入 s 分别怎么写？为什么顺序不能乱？
3. 有序表删除 x 为什么让 p 停在前驱？&& 的前后顺序为什么不能换？
4. 循环链表遍历为什么要比较“是否回到头结点”？逆转后头结点的指针指向谁？
5. 快慢指针为什么能找到中点？回文判断的时间、空间复杂度各是多少？

到这里，我们已经解决了什么？下一步还缺什么？

我们用指针把“排成一排”的数据串了起来，并学会了增、删、逆置。**但表在一端或两端按固定规则存取时，可以更简单、更高效**——这就是下一章的栈和之后的队列。后续联系：链表是链式栈/队列、哈希表拉链、邻接表（图）、LRU 缓存的基础；循环链表用于轮转调度。

第 3 章　栈：后进先出

本章回答 为什么有些问题天然要“最近的先处理”？栈怎样定义、怎样实现？括号为什么用栈判断？给定入栈次序，哪些出栈序列合法、栈要多深？（对应试卷：选择 2、3、4，填空 1、5，简答 3。）

3.1 为什么需要栈

叠放的盘子，最后放上去的最先被拿；浏览器“后退”，先回到最近看的那页；“撤销”，先撤销最近的操作；函数 A 调用 B、B 又调用 C，C 先结束、再回到 B、最后回到 A。这些场景有共同规律：**“最近的先处理”**。线性表本来可以在任意位置存取，现在**把存取限制在同一端**，就得到栈——限制越多，保证越强，实现越简单。

栈（stack）

栈是**只允许在表的一端（栈顶）进行插入和删除**的线性表。允许操作的一端称**栈顶**，另一端称**栈底**。插入叫**入栈（push）**，删除叫**出栈（pop）**。元素的进出顺序是**后进先出**（Last In First Out，**LIFO**）。

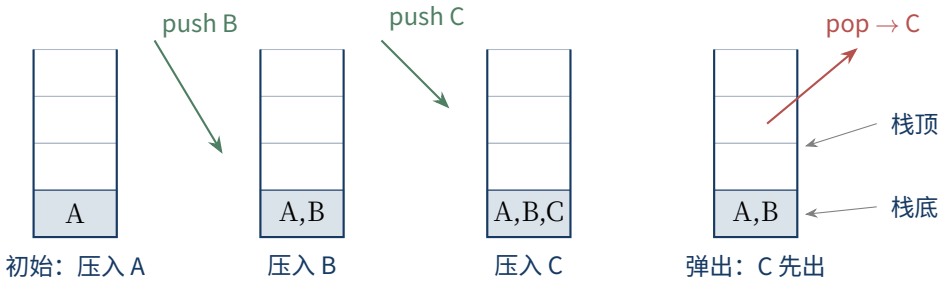


图 3-1　栈的入栈与出栈：每次只能在栈顶操作，最后进去的 C 第一个出来（LIFO）。

对应试卷：选择题 2 与 填空题 1 [原图识别]

选择 2：“元素出栈的操作是在栈的哪个端进行的？”——定义里写着“只在一端（栈顶）操作”，所以 **A（栈顶）**；B、D 都含栈底，错误。　**填空 1**：栈的进出顺序是 **后进先出（LIFO）**；队列是 **先进先出（FIFO）**（下一章）。

3.2 栈的实现与复杂度（教学补全）

用数组 `data[0..M-1]` 加一个整数 `top`（栈顶元素下标，空栈为 -1）：

```
bool push(int x){ if (top == M-1) return false;  data[++top] = x; return true; } // 满：上溢
bool pop(int &x){ if (top == -1)  return false;  x = data[top--]; return true; } // 空：下溢
```

push、pop、取栈顶都只动 top 和一个元素，**时间** $O(1)$。栈也可以用链表实现（**链栈**：栈顶就是链表头，入栈=头插，出栈=删头结点），同样 $O(1)$ 且没有容量上限。

3.3 括号匹配为什么用栈（选择 3）

问题：判断表达式中的括号 ( ) [ ] { } 是否配对且正确嵌套，如 {[()]} 对，([)] 错。

想法：从左到右扫描。遇到**左括号**，它在等待配对，先“存起来”；遇到**右括号**，它要匹配的一定是**最近一个还没配对的左括号**——而“最近存的”正好就是栈顶。

算法

对字符串逐字符：左括号入栈；右括号则看栈——若栈空或栈顶不是对应的左括号，则**不匹配**，否则弹出栈顶；扫描完毕栈**必须为空**才匹配（否则有未配对的左括号）。时间 $O(n)$。

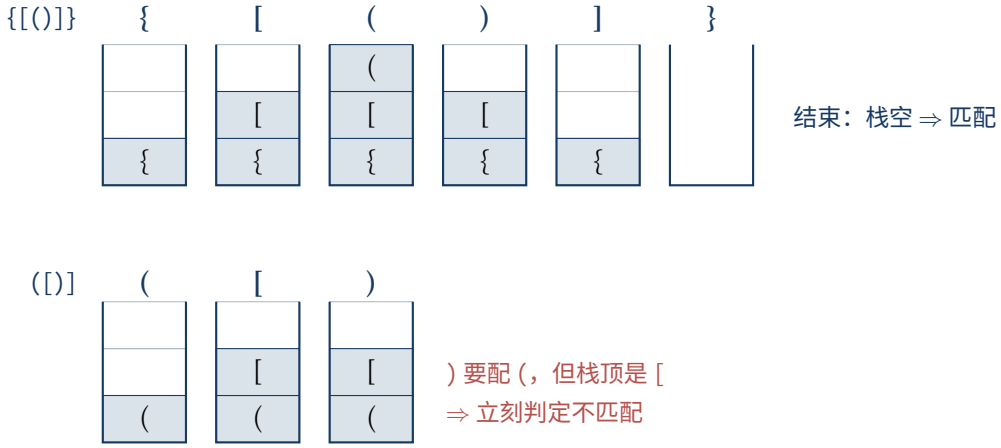


图 3-2　上：{[()]} 逐步入栈、出栈，最后栈空，匹配。下：([)] 在读到) 时栈顶是 [，立刻判定错误。

对应试卷：选择题 3 [原图识别]

“判断表达式中左右括号是否配对，采用（　）数据结构最佳。” 顺序表只能存放，没有“与最近的左括号配对”的机制；队列是“最早的先配”，与括号嵌套规则相反（([)] 就会被误判为对）；矩阵无关。 答案 **B（栈）**。

易错点：不要用“计数器”代替栈

只数“左括号个数 − 右括号个数”能判断 (()，却分不清 ([)] 这类**种类交叉**的错误。栈保存的是“哪一种左括号最近未配对”，信息比计数多。

3.4 出栈序列：哪些合法，栈要多深（选择 4、简答 3、填空 5）

入栈次序固定后，进栈、出栈可以**交错进行**，所以出栈序列不止一种。例：入栈次序 $1,2,3$，可以全部入栈再全出得到 $3,2,1$；也可以入 1 出 1、入 2 出 2、入 3 出 3 得到 $1,2,3$；**但** $3,1,2$ **不可能**：3 先出时 1、2 都还在栈里，2 压在 1 上面，1 不可能先于 2 出栈。

合法出栈序列的判据（教学补全）

入栈次序为 $1,2,\ldots,n$，则出栈序列合法 $\iff$ 其中**不存在三个元素** $i<j<k$ **以“**k,i,j**”的先后顺序出栈**（通常称“312 模式”）。合法序列的总数是卡特兰数 $C_n=\dfrac{1}{n+1}\binom{2n}{n}$（$n=3$ 时为 5，$n=5$ 时为 42）。

为什么？（要证两个方向）

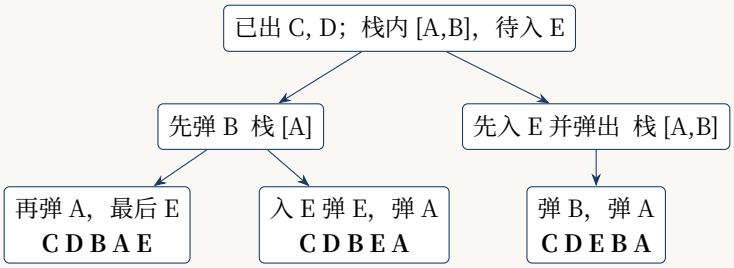
必要性：若 k 先于 i,j 出栈（$i<j<k$），则在 k 出栈那一刻，i,j 早已入栈（先于 k）却尚未出栈，故都在栈内，且 j 在 i 之上。此后 i 想出栈必须先弹出 j，所以只可能 j 先于 i 出栈，与“i,j”顺序矛盾。 **充分性**：用“按需入栈”模拟：要输出 y 时，若 y 还没入栈就依次入栈到 y，若 y 已在栈内就必须在栈顶。若模拟卡住，说明 y 在栈内却被某个 t 压住（$y<t$，y 先入栈，t 后入栈、且 t 尚未出栈）；t 之所以已入栈，是因为更早输出的某个 $z>t$ 需要它让路。于是输出次序是 z,y,t，而入栈次序 $y<t<z$——恰好是 312 模式，矛盾。 □

栈要多深？栈的**容量至少要等于整个过程中栈内元素个数的最大值**（峰值）。由于“元素出栈即进入队列”时队列只是原样保存出栈顺序，所以**队列的出队序列 = 栈的出栈序列**，填空 5 就是在求这个峰值（见图 S-1，结果 4）。

对应试卷：简答 3 [原图识别]

题：进栈次序 A,B,C,D,E，C 第一个出、D 第二个出，有哪些出栈次序?

思路：C 要第一个出，必须先把 A、B、C 全压进去再弹出 C；D 紧接着第二个出，D 必须在 C 弹出后压入并立刻弹出。此时栈中剩 A、B（B 在上），只剩 E 未入栈。后面怎样安排仅取决于 E 什么时候入栈:



共 **3 种**：C,D,E,B,A；C,D,B,E,A；C,D,B,A,E。（程序穷举 $5!=120$ 个排列验证：恰好这 3 个合法。）

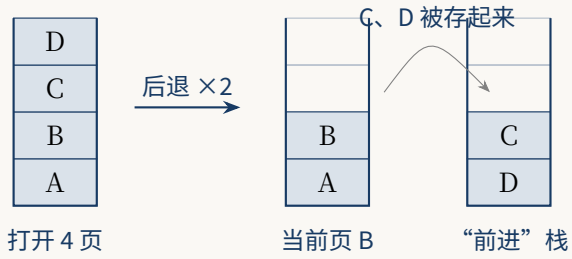
图 3-3　简答 3 的决策树：在栈内剩 [A,B] 时，下一步“弹 B”还是“入 E”，决定了三种结果。

对应试卷：选择题 4（题意不完整，按补全版本）[原图识别] [教学补全]

补全后的题意：输入序列为 $P_1, P_2, \ldots, P_n$，输出序列恰为 $1, 2, \ldots, n$，且 $P_3 = 3$，问 P_1 的值。
解法：输出第一个必须是 1，第二个是 2。(i) $P_1 = 1$：压 1 弹 1，压 $P_2 = 2$ 弹 2，压 3 弹 3，可行；(ii) $P_1 = 2$，$P_2 = 1$：压 2、压 1、弹 1、弹 2，压 3 弹 3，可行；$n \geq 4$ 时还有 $P_1 = 4$（输入 $4, 1, 3, 2$：压 4、压 1、弹 1、压 3、压 2、弹 2、弹 3、弹 4）。所以 P_1 **可以是 2，但不一定是 2，也不一定是 1**。 故“可能是 2”正确，“一定是 1”“一定是 2”均错，答案 **A**。（**注意**：原题被划掉的题干无法辨认，这只是对补写版本的解答。）

生活中的它 · 浏览器的“后退”键

场景：依次打开页面 A、B、C、D，然后连按两次“后退”。
对应：已访问页面 ↔ 栈中元素；打开新页面 ↔ push；后退 ↔ pop（回到栈顶的“上一个”）。
小判断：打开 A,B,C,D 后栈为 $[A, B, C, D]$（D 在顶）；后退一次，栈 $[A, B, C]$，当前页 C；再退一次，栈 $[A, B]$，当前页 B。
边界：真实浏览器还有“前进”（再用一个栈存放刚被弹出的页面），并且在后退后打开新页面会清空“前进”栈。



易错点

(1) “入栈次序固定”不等于“出栈次序固定”，进栈出栈可交错。 **(2)** 求“栈至少多大”要看**整个过程的最大深度**，不是某一个时刻、也不是元素总数。 **(3)** 栈空时 pop（下溢）、栈满时 push（上溢）要先判断。 **(4)** 链栈的“栈顶”是链表头，不是表尾。

对照：栈 ↔ 函数调用栈与递归（思想重现型）

一句话定位：程序运行时，系统用一块叫**调用栈**的内存保存“每一次函数调用的现场”（参数、局部变量、返回地址）——递归调用本质上就是系统替你用了一个栈。

维度	数据结构中的栈	函数调用栈
存放什么	程序员指定的元素	每次调用的栈帧（参数、局部变量、返回地址）
谁来操作	代码中显式 push / pop	调用函数时系统自动压栈，返回时自动弹栈
顺序规则	LIFO	LIFO：最后被调用的函数最先返回
容量	数组容量或内存	系统规定上限，递归过深会**栈溢出**

栈的结论	迁移到调用栈	理由或反例
后进先出	✓	先调用的函数要等被它调用的函数全部返回后才返回
任何递归都能改写成“显式栈 + 循环”	✓	把调用栈搬到自己的数组里
“栈满时 push 返回 false，可以处理”	×	调用栈满了就是程序崩溃（栈溢出），没有机会处理
栈内元素可被程序随意查看	△	调用栈中间的栈帧通常不能直接访问，只有调试器才能看

同一个小算例：计算 $3!$。递归：`fact(3)` 调用 `fact(2)`，再调用 `fact(1)`；栈深依次 $1,2,3$，再依次返回 $1,2,6$。显式栈：入栈 3、2、1，再依次弹出相乘 $1\times2\times3=6$。两种做法栈深峰值都是 3。

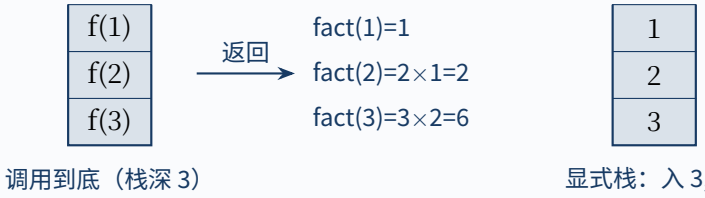


图 3-4　递归求 3! 的调用栈（左）与显式栈（右）：同样的 LIFO，同样的峰值 3（f(n) 表示 fact(n)）。

学完这里，你应该能回答

1. 栈的定义和 LIFO 是什么意思？push、pop 作用在哪一端？
2. 括号匹配时为什么“最近的未配对左括号”正好是栈顶？为什么结束时栈必须为空？
3. 入栈次序 $1\sim n$ 时，怎样判断一个出栈序列是否合法（312 模式）？
4. “栈至少容纳几个元素”怎么求？为什么队列出队序列等于栈出栈序列？
5. 栈与函数调用栈有什么相同、什么不同？

到这里，我们已经解决了什么？下一步还缺什么？

栈解决了“最近的先处理”。但排队、缓冲区、广度优先遍历需要的是**“最早的先处理”**——两端都要用。下一章的队列把“插入在一端、删除在另一端”固定下来，并解决数组实现中的“假溢出”。后续联系：栈在表达式求值、深度优先搜索、回溯（八皇后）、编译器语法分析中不断出现。

第 4 章　队列：先进先出

本章回答　队列和栈只差在“从哪一端出”，为什么这么一改就适合排队？用数组实现队列会遇到什么“怪事”（假溢出）？循环队列怎样解决，为什么入队要写成 `rear=(rear+1) mod (m+1)`？　（对应试卷：选择 5，填空 1。）

4.1　为什么需要队列

银行叫号、打印任务、客服热线、消息推送……都要求**“先来先服务”**。栈是“后来的先走”，显然不行；我们需要在**一端放进去、另一端取出来**。

队列（queue）

队列是**只允许在一端（队尾 rear）插入、在另一端（队头 front）删除**的线性表。插入叫**入队（enqueue）**，删除叫**出队（dequeue）**。元素的进出顺序是**先进先出**（First In First Out，**FIFO**）。

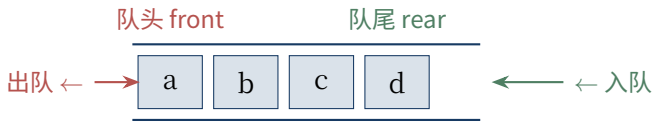


图 4-1　队列：左端（队头）出，右端（队尾）进，先进去的 a 先出来。

相近概念辨析	栈	队列
进出规则	后进先出（LIFO）	先进先出（FIFO）
插入/删除位置	同一端（栈顶）	不同端（队尾进、队头出）
典型应用	括号匹配、函数调用、撤销	排队、广度优先、层次遍历、缓冲区
指针个数	1 个（top）	2 个（front、rear）

4.2 数组实现的麻烦：假溢出

用数组 A[0..N-1]，front 指队头元素，rear 指队尾元素的**下一个空位**：入队写 A[rear] 再 rear++；出队 front++。反复入队出队后，两个指针都在向右“漂移”：

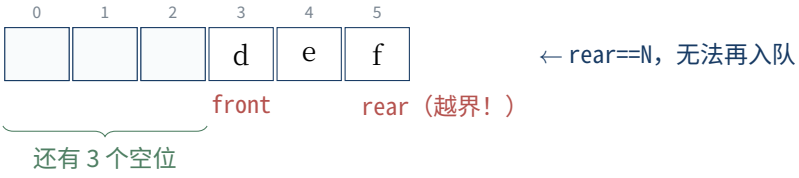


图 4-2　假溢出：数组前面明明有 3 个空位，却因为 rear 已到末尾而“满了”。

假溢出就是：队列并不真满，而是指针移到了数组末端。解决办法：**把数组首尾想象成连在一起的环**。

4.3 循环队列

循环队列的关键公式

设数组长度为 N（下标 $0, 1, \ldots, N-1$），front 指队头元素，rear 指**下一个入队位置**。

入队：A[rear]=x; rear=(rear+1) mod N

出队：x=A[front]; front=(front+1) mod N

队空：front==rear

队满（空一格）：(rear+1) mod N==front

元素个数：$\boxed{(\text{rear} - \text{front} + N) \bmod N}$

符号：N 为数组单元总数；mod 取余数，使下标超过 $N-1$ 时“绕回” 0。**条件**：队空与队满不能都用 front==rear，故常**空出一个单元**，此时最多存放 $N-1$ 个元素。

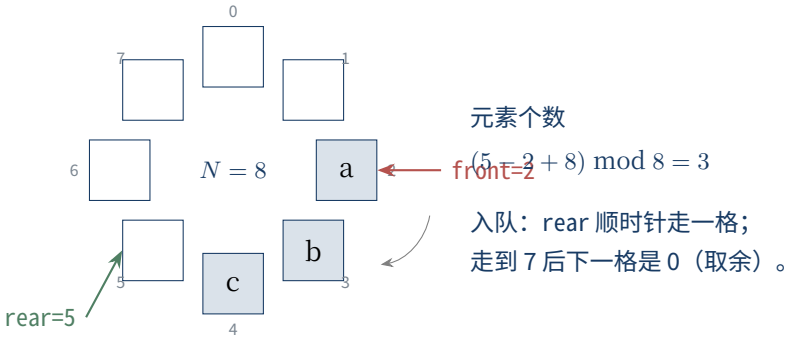


图 4-3　$N = 8$ 的循环队列：蓝格是队内元素 a,b,c，front 指队头，rear 指下一个空位。

为什么元素个数是 $(\texttt{rear} - \texttt{front} + N) \bmod N$？（教学补全）

从 front 顺时针数到 rear 要走的格数就是元素个数。若 rear $\geq$ front，没有绕圈，格数 = rear − front；若 rear < front，绕过了数组末尾，格数 = rear + N − front。两种情形统一写成 $(\texttt{rear} - \texttt{front} + N) \bmod N$：第一种情形加 N 后再取余恰好还原为 rear − front。 □

对应试卷：选择题 5 [原图识别]

“循环队列存储在数组 $A[0..m]$ 中，则入队时队尾指针 rear 的操作为？”

已知数组下标从 0 到 m，共有 $m + 1$ 个单元，所以**模数应是单元总数** $N = m + 1$。 **判断各选项**（图 4-4）：

A `rear=rear+1`：到 m 后继续加 1 就越出数组，没有“循环”； B `mod (m-1)`：下标只能取 $0 \sim m - 2$，$A[m-1], A[m]$ 永远用不到； C `mod m`：下标只能取 $0 \sim m - 1$，最后一个单元 $A[m]$ 用不到； **D `mod (m+1)`**：下标 $0 \sim m$ 全部可达，正确。答案 **D**。



图 4-4　取 $m = 5$（数组 $A[0..5]$，6 个单元）时四个选项能“到达”的下标（蓝色）。只有 D 恰好覆盖整个数组且不越界。

生活中的它 · 银行叫号机

场景：取号机发 0–99 号，号码用完回到 0；叫号屏显示当前叫到的号。

对应：发号 ↔ 入队（rear 加 1）；叫号 ↔ 出队（front 加 1）；“99 的下一个是 0” ↔ mod 100。

小估算：当前叫到 97 号，最新发出的是 2 号，则下一个发出的号码是 3（rear= 3）；还没被叫到的人数 $(3 - 97 + 100) \bmod 100 = 6$ 人（97、98、99、0、1、2 号）。

边界：真实银行有 VIP 窗口、预约号，不再是纯 FIFO，那属于“优先队列”。

易错点

(1) 模数是**数组单元总数**，不是最大下标：$A[0..m]$ 要 `mod (m+1)`。 **(2)** 空出一个单元的方案中，队满条件是 `(rear+1) mod N == front`，不是 `rear==front`（那是队空）。 **(3)** 容量 N 的循环队列最多存 $N - 1$ 个元素（“空一格”法）；用计数器 `size` 或标志位可存满 N 个。 **(4)** 链队列没有“假溢出”，也不用取模：入队接在尾结点后，出队删头结点。

对照：队列 ↔ 排队论中的“队列”（类比同构型 / 同名异义型）

一句话定位：数据结构的队列是**存放元素的容器**，规定“先进先出”；运筹学/概率论里的排队论研究的是**随机到达、需要服务的系统**，队列只是其中的一部分。二者用的是同一个词，关注的问题却不同。

维度	数据结构：队列	排队论：排队系统
研究对象	元素的存放与存取次序	到达、等待、服务的统计规律
核心量	`front`、`rear`、元素个数	到达率 λ，平均停留时间 W，平均队长 L
次序	FIFO（定义）	可以是 FIFO、优先、随机等
容量	数组大小 N（循环队列至多 $N-1$）	可有无限或有限缓冲区

旧知识中的结论	迁移	理由或反例
先到的先被服务	△	仅在“服务次序为 FIFO”时成立；排队论还研究优先级、随机次序
稳态下平均队长 $L=\lambda W$（利特尔定律）	△	对**稳定**的系统成立，与到达分布无关；可用来估计缓冲区大小
容量有限、满了就拒绝入队	✓	对应有限缓冲排队系统的“丢包”
数据结构中的队列有“随机到达率”	×	数据结构只规定存取规则，不含概率模型

同一个小算例：到达率 $\lambda=2$ 个/秒，每个元素平均在队列里停留 $W=3$ 秒。排队论：平均队长 $L=\lambda W=6$ 个。数据结构视角：如果你要给这个队列选数组大小，平均占用约 6 个，为应付波动可再留余量，如取 $N=16$。

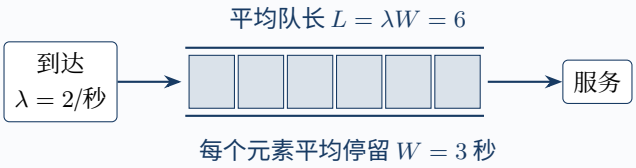


图 4-5　排队系统：到达率、队长、停留时间之间的利特尔定律 $L=\lambda W$。

学完这里，你应该能回答

1. 队列的定义是什么？进、出分别在哪一端？它与栈有什么不同？
2. 什么叫假溢出？循环队列怎样避免？
3. 数组 $A[0..m]$ 的循环队列，入队后 `rear` 怎样更新？为什么模数是 $m+1$？
4. 队空、队满、元素个数分别怎样判断（空一格方案）？
5. 链队列为什么没有假溢出？

到这里，我们已经解决了什么？下一步还缺什么？

前四章研究的都是“一维排成一排”的结构。但文件夹、公司架构、表达式、家谱都是一**个对象下面挂着多个对象**的层次关系——线性表装不下，需要树。后续联系：队列是树的层次遍历、图的广度优先搜索、操作系统进程调度与消息队列的基础。

第 5 章　树与二叉树

本章回答 为什么要把“排成一排”升级成“一对多”的树？二叉树有哪些必须会推的性质？四种遍历、由遍历序列还原树、森林转二叉树、表达式树，各是怎么回事？（对应试卷：选择 1、7、8、9，填空 4，简答 4、5，编程 2。）

5.1 为什么需要树，树里有哪些名词

文件夹下面有子文件夹，子文件夹下面还有文件；公司有部门、部门有小组；家谱里一个人有多个孩子。这些都是**“一个对象下面挂着若干个对象”**的层次关系。线性表里每个元素最多只有一个后继，描述不了，于是引入**树**。

树（tree）

树是 $n\ (n \geq 0)$ 个结点的有限集。$n = 0$ 称**空树**；$n > 0$ 时：**有且仅有一个**特定结点叫**根**；其余结点分成 $m\ (m \geq 0)$ 个**互不相交**的集合 $T_1, \ldots, T_m$，每个集合本身又是一棵树，称为根的**子树**。——这是一个**递归**定义：树由根和若干更小的树构成。

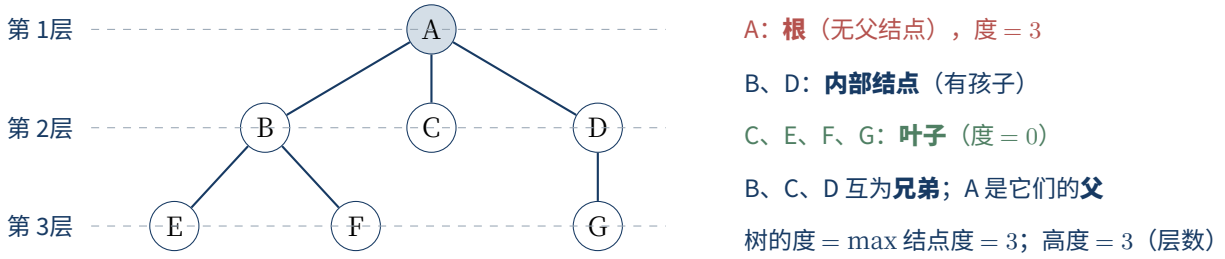


图 5-1　一棵树及术语。结点的**度**=孩子个数；**层**从根的第 1 层数起；树的**高度/深度**=最大层数。

约定提示：“高度”有两种数法

教材有两种约定：**按层数**（根在第 1 层，只有一个结点的树高度为 1）；**按边数**（根的高度为 0）。本课程试卷的选择题 7、8 使用**层数约定**（选择 7 的“深度为 k 的完全二叉树最少 2^{k-1} 个结点”只有在根为第 1 层时才成立）。看到题目先弄清约定。

5.2 二叉树：每个结点至多两个孩子，且分左右

二叉树（binary tree）

二叉树是 $n\ (n \geq 0)$ 个结点的有限集，或者为空，或者由一个根结点和**两棵互不相交**的、分别称为**左子树**和**右子树**的二叉树组成。**注意：二叉树的子树有左右之分，即使只有一个孩子也要说明是左还是右。**

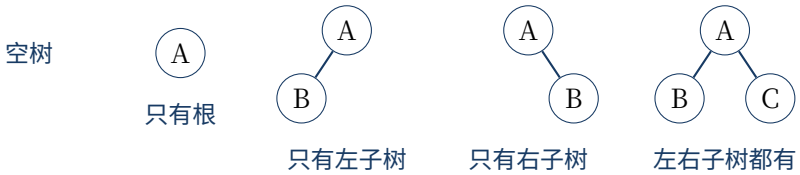


图 5-2　二叉树的 5 种基本形态。第 3、4 种是**不同**的二叉树（左右有别），虽然都只有两个结点。

二叉树的基本性质（以根为第 1 层）

性质 1　第 i 层至多有 2^{i-1} 个结点；

性质 2　深度为 k 的二叉树至多有 $2^k - 1$ 个结点；

性质 3 设叶子数为 n_0，度为 2 的结点数为 n_2，则 $\boxed{n_0 = n_2 + 1}$；

性质 4 具有 n 个结点的完全二叉树深度为 $\lfloor \log_2 n \rfloor + 1$；深度为 k 的完全二叉树**最少** 2^{k-1} 个、**最多** $2^k - 1$ 个结点；

性质 5 **编号规则**（完全二叉树按层次、同层从左到右编号）：若根编号为 **0**，则编号 i 的结点**左孩子** $2i+1$，**右孩子** $2i+2$，**父结点** $\lfloor (i-1)/2 \rfloor$；若根编号为 1，则为 $2i$、$2i+1$、$\lfloor i/2 \rfloor$。

满二叉树与完全二叉树

满二叉树：深度为 k 且有 $2^k - 1$ 个结点（每层都满）。**完全二叉树**：深度为 k、结点按层次从上到下、同层从左到右编号后，与满二叉树中**前** n **个**结点位置一一对应——直观地说，“只有最后一层可以不满，且最后一层的结点都靠左”。满二叉树一定是完全二叉树，反之不然。

性质的证明（教学补全）

性质 1（对 i 归纳）：$i = 1$ 时只有根，$2^0 = 1$；若第 i 层至多 2^{i-1} 个，每个结点至多 2 个孩子，则第 $i+1$ 层至多 $2 \cdot 2^{i-1} = 2^i$ 个。 **性质 2**：各层最多的和 $\sum_{i=1}^{k} 2^{i-1} = 2^k - 1$（等比数列求和）。

性质 3：设度为 1 的结点数为 n_1，总结点数 $n = n_0 + n_1 + n_2$。另一方面，除根外每个结点恰有一个“进来的分支”，分支总数 $= n - 1$；而分支也等于各结点“出去的分支”之和 $= n_1 + 2n_2$。于是 $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$，化简得 $n_0 = n_2 + 1$。 □

性质 4：前 $k-1$ 层是满的，共 $2^{k-1} - 1$ 个结点；第 k 层至少有 1 个，故至少 2^{k-1} 个（选择 7 的依据）；第 k 层最多放满，共 $2^k - 1$ 个。由 $2^{k-1} \le n \le 2^k - 1$ 取对数得 $k - 1 \le \log_2 n < k$，即 $k = \lfloor \log_2 n \rfloor + 1$。

性质 5：层次编号恰是“广度优先”的顺序：结点按编号依次出队，各出队结点的孩子按左、右顺序入队，因此**孩子的编号次序与父结点的编号次序一致**。编号为 i 的结点有孩子，说明它前面的 i 个结点（编号 $0..i-1$）在完全二叉树中**都有两个孩子**（否则靠左排列就轮不到 i 有孩子），这 $2i$ 个孩子加上根，占用了编号 $0..2i$，所以 i 的左孩子是 $2i+1$（填空 4 的依据），右孩子是 $2i+2$，父结点为 $\lfloor (i-1)/2 \rfloor$。 □

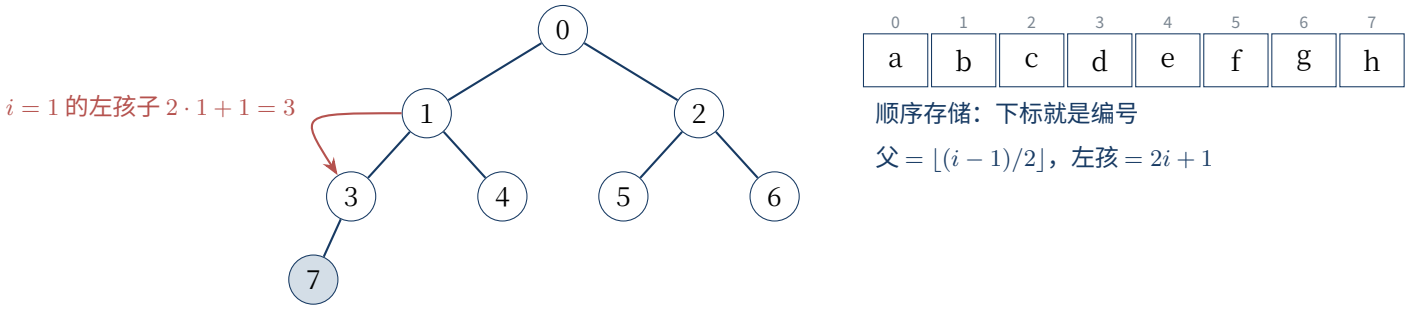


图 5-3　完全二叉树从 0 编号。结点 1 的左孩子是 3（$2 \cdot 1 + 1$）；用数组存放时，下标即编号，因而完全二叉树不需要指针。

对应试卷：选择 7 与 填空 4 [原图识别]

选择 7：深度为 k 的完全二叉树最少有几个结点？前 $k-1$ 层满（$2^{k-1} - 1$ 个）+ 第 k 层至少 1 个 $= 2^{k-1}$，选 **B**。检验：$k = 3$，最少 4 个（根、两个孩子、一个孙子靠左），$2^2 = 4$ √；A 的 $2^{k-1} + 1$ 比最少值多 1；C、D 太小，连前 $k-1$ 层都放不下。

填空 4：编号从 0 起，i 的左孩子编号为 $2i+1$（如 $i = 0$ 时左孩子是 1，$i = 1$ 时是 3）。若误用“从 1 编号”的公式 $2i$，会得到 0 的左孩子为 0，自相矛盾。

5.3 二叉树的四种遍历

遍历就是按某种规律把每个结点**恰好访问一次**，把非线性的树“拉成”一个线性序列。

先序、中序、后序、层序

先序（根左右）：访问根，先序遍历左子树，先序遍历右子树。 **中序（左根右）**：中序遍历左子树，访问根，中序遍历右子树。 **后序（左右根）**：后序遍历左子树，后序遍历右子树，访问根。 **层序**：从上到下、同层从左到右依次访问（需要**队列**）。

```
void PreOrder(Node *t){ if (t) { visit(t); PreOrder(t->leftChild); PreOrder(t->rightChild); } }
void InOrder (Node *t){ if (t) { InOrder (t->leftChild); visit(t); InOrder (t->rightChild); } }
void PostOrder(Node *t){ if (t) { PostOrder(t->leftChild); PostOrder(t->rightChild); visit(t); } }
```

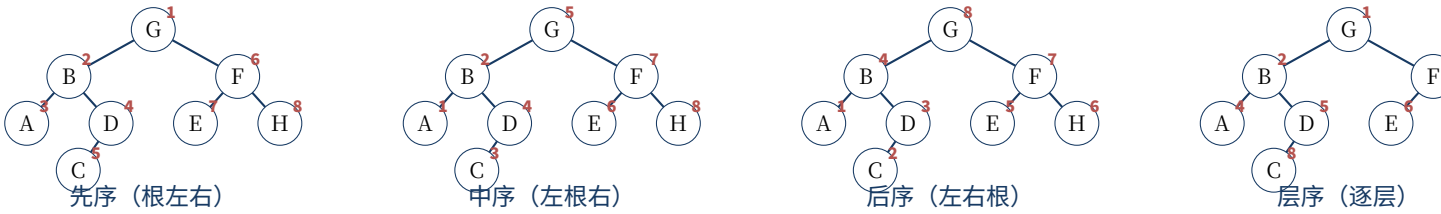


图 5-4　同一棵树（试卷简答 5 的树）的四种遍历，红色数字是访问次序。先序 GBADCFEH；中序 ABCDGEFH；后序 ACDBEHFG；层序 GBFADEHC。

层次遍历算法与复杂度（编程题 2，教学补全）

想法：层序遍历要求“先进入的结点，它的孩子也先被访问”，这正是 FIFO。**做法**：根入队；只要队列非空：出队一个结点并访问，再把它的左、右孩子（若非空）依次入队。

```
void BinaryTree<ElemType>::LevelOrder(void (*visit)(const ElemType &)) const {
  LinkQueue<const BinTreeNode<ElemType> *> q;      // 队列，元素是结点指针
  const BinTreeNode<ElemType> *cur;
  if (root != NULL) q.InQueue(root);               // 根非空则入队
  while (!q.Empty()) {                             // 还有结点没访问
    q.OutQueue(cur);                               // 出队，得到当前结点
    (*visit)(cur->data);                           // 访问
    if (cur->leftChild  != NULL) q.InQueue(cur->leftChild);   // 左孩子入队
    if (cur->rightChild != NULL) q.InQueue(cur->rightChild);  // 右孩子入队
  }
}
```

正确性：用归纳法，每个结点入队一次、出队一次；结点出队时，上一层的所有结点都已出队（其孩子已在它之前入队），所以同层从左到右、逐层向下。**时间**：每个结点入队出队各 1 次，$O(n)$；**空间**：队列中最多容纳一层的结点，最坏 $O(n)$。参考答案与此完全一致。



图 5-5　层序遍历前几步的队列变化（对简答 5 的树）：队列始终保存“已发现、未访问”的结点。

5.4 由遍历序列还原二叉树（简答 5）

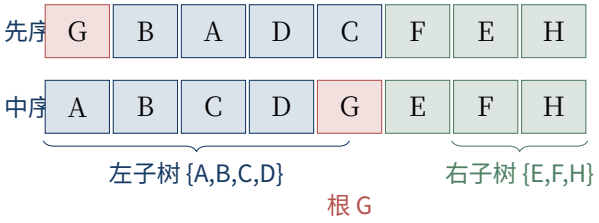
先序 + 中序 ⇒ 唯一确定二叉树

先序序列的**第一个**元素是根；在中序序列中找到它，**左边是左子树的中序、右边是右子树的中序**；左、右部分的长度又告诉我们先序序列中哪一段属于左子树、哪一段属于右子树；对两边**递归**。后序序列的**最后一个**元素是根，用法相同。

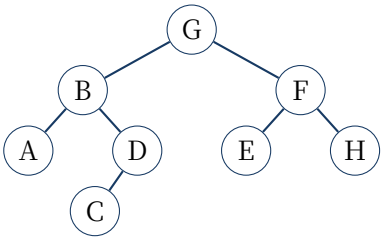
唯一性与反例（教学补全）

唯一性（对结点数 n 归纳）：$n = 0, 1$ 显然。设结点数 $< n$ 时成立。先序第一个是根 r（唯一）；r 在中序序列中的位置唯一（结点互不相同），它把中序分成左子树的 k 个元素和右子树的 $n-1-k$ 个元素，从而先序中紧跟 r 的 k 个元素属于左子树；左右子树的“先序 + 中序”结点数更少，由归纳假设各自唯一，所以整棵树唯一。 □

为什么必须有中序？ 只给先序和后序不一定唯一：先序 AB、后序 BA 对应**两棵**不同的二叉树（B 是 A 的左孩子，或是右孩子），因为它们都告诉你“A 是根、B 在下面”，却没法分左右。



第 1 步：G 是根；
左 4 个、右 3 个
第 2 步：左部分先序 B A D C，根 B；
中序 A|B|C D ⇒ 左 {A}，右 {C,D}……



后序 = A C D B E H F G
（左子树 ACDB，右子树 EHF，再根 G）

图 5-6　由先序 GBADCFEH 与中序 ABCDGEFH 还原：先找根 G，把中序分成左 4、右 3，再对左右部分递归，得到下方的树（D 的左孩子是 C）。

5.5 先序与中序的“栈”关系（选择 9）

中序序列是先序序列的一个出栈序列

对二叉树做**非递归**中序遍历：沿左链往下走，**经过的结点依次入栈**（入栈顺序恰是**先序**）；走不动时弹出栈顶并访问（弹出顺序恰是**中序**）。所以：**若先序序列为 $1, 2, \ldots, n$，则合法的中序序列恰是以 $1, 2, \ldots, n$ 为入栈次序的合法出栈序列**（第 3 章的 312 判据同样适用）。

对应试卷：选择 9 [原图识别]

先序为 1,2,3,4,5。四个候选中序序列，哪个**不可能**？用入栈 1..5 去“产生”它们：
A **3,2,1,4,5**：入 1,2,3，弹 3,2,1，入 4 弹 4，入 5 弹 5 √；　B **2,3,1,4,5**：入 1,2，弹 2，入 3 弹 3，弹 1，入 4 弹 4，入 5 弹 5 √；　C **2,1,4,5,3**：入 1,2，弹 2,1，入 3，入 4 弹 4，入 5 弹 5，弹 3 √；　**D 2,1,5,3,4**：入 1,2，弹 2,1，入 3,4,5，弹 5，**此时栈顶是 4，不是 3**，不能弹 3 ☒（在 312 判据下：3,4,5 入栈顺序递增，出栈为 5,3,4 构成“k, i, j”）。 答案 **D**。

5.6 表达式树与并行计算时间（选择 8）

表达式树：叶子是操作数，内部结点是运算符，一个内部结点对应一次运算，左右子树分别是它的两个操作数。如 a+b*c 的树：根 +，左孩子 a，右孩子 *（它的孩子是 b、c）。该树的**后序遍历**就是后缀表达式 a b c * +。

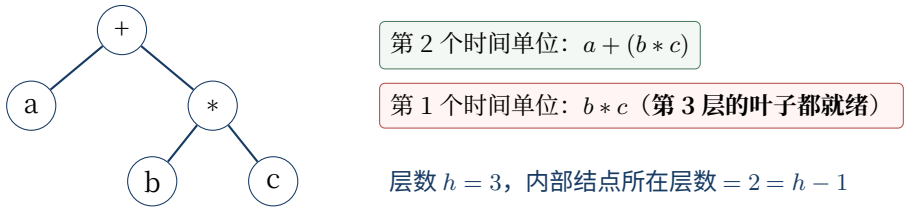


图 5-7　a+b*c 的表达式树。有足够多的 ALU 时，每个“时间单位”可以同时完成同一层的所有运算。

为什么最短时间 = 高度 −1（教学补全）

一次运算必须等它的两个操作数都就绪。把内部结点按“到叶子的最长距离”分层：最下面一层的运算只依赖叶子，可以立刻并行完成；上一层的运算依赖下一层结果……所以**完成时间 = 内部结点的层数 = 树的层数 −1**（叶子层不需要运算）。每个时间单位至多“推进一层”，所以不可能更快；有足够多 ALU 就能做到。**注意**这里“高度”按层数计（根为第 1 层）；若按边数计，答案就应是“树的高度”本身。选 A。

5.7 森林与二叉树的转换（简答 4）

一般的树每个结点可以有任意多个孩子，用链表表示很不方便（每个结点要放不定个数的指针）。**“左孩子—右兄弟”表示法**把任何树/森林都变成二叉树，结点只需两个指针。

转换规则

树 → 二叉树：(1) 同一父结点的兄弟之间加连线；(2) 每个结点**只保留与第一个孩子的连线**，删去与其他孩子的连线；(3) 以根为轴，把**第一个孩子**放在**左**侧（左指针），把**下一个兄弟**放在**右**侧（右指针）。

森林 → 二叉树：先把每棵树各自转换，再把**各树的根按顺序连成“兄弟”**（后一棵树的二叉树作为前一棵根的右子树）。

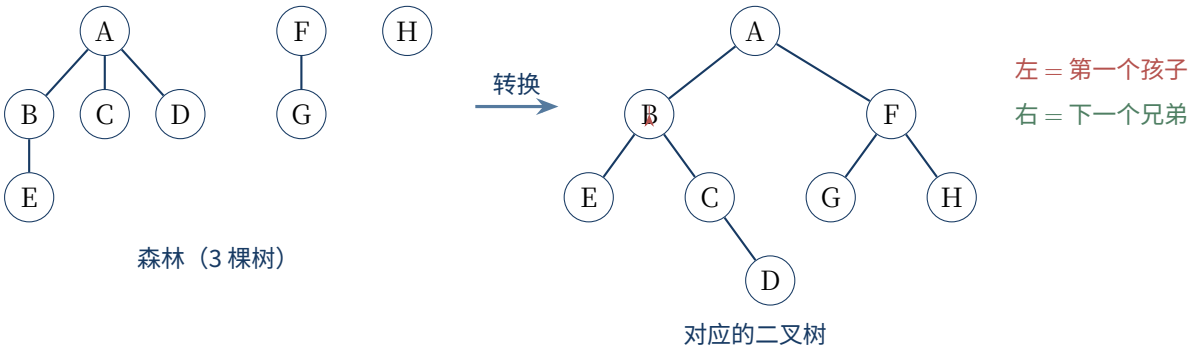


图 5-8　试卷简答 4 的森林转二叉树：A 的左孩子是 B（第一个孩子）；B 的右孩子是 C，C 的右孩子是 D（兄弟链）；A 的右孩子是 F（下一棵树的根），F 的左孩子是 G，右孩子是 H。

易错点：二叉树 → 森林也要会（逆变换）

看二叉树里的**右链**：沿右指针一直走下去的所有结点互为兄弟。根的右链上有几个结点，森林里就有几棵树；每个结点的左子树对应它的孩子们。如图 5-8 的二叉树：根 A 的右链是 A、F、H，所以有 3 棵树。

生活中的它 · 电脑里的文件夹

场景：磁盘的“文件夹—子文件夹—文件”。
对应：根目录 $\leftrightarrow$ 根；子文件夹 $\leftrightarrow$ 孩子；文件 $\leftrightarrow$ 叶子；“路径” $\leftrightarrow$ 从根到某结点的唯一一条路。
小估算：若每个文件夹至多放 2 个子文件夹（二叉），深度 10 层最多 $2^{10}-1=1023$ 个文件夹；反过来，装 1000 个文件夹的完全二叉形状，层数仅 $\lfloor \log_2 1000 \rfloor + 1 = 10$——层次结构让“找一个文件”只需沿着约 10 步走下去。
边界：真实文件夹的子文件夹数量不限（一般树，不是二叉树），而且有“快捷方式”会形成环，那就不再是树而是图。

对照：树 $\leftrightarrow$ 图（推广型）

一句话定位：树是**连通且没有回路的图**；图是更一般的“点 + 线”结构。树的结论很多可以搬到图，但“遍历时不用标记”这一条**不能**。

维度	树	图
结点间关系	一对多（父 $\rightarrow$ 子）	多对多
有无回路	无	可以有
边数	结点数 -1	不确定
两点间的路径	恰有一条	可以没有、一条或多条
遍历方式	先/中/后序（深度优先）、层序（广度优先）	深度优先搜索 DFS、广度优先搜索 BFS

树的结论	迁移到图	理由或反例
层序遍历用队列、逐层向外	△	即 BFS，但要用“已访问”标记，否则遇到回路会重复入队
深度优先遍历可用递归	△	即 DFS；同样必须标记已访问，否则在回路里死循环
“遍历时不需要标记已访问”	×	图中 A–B–C–A 若不标记，会反复绕圈
“任意两点之间路径唯一”	×	有回路的图两点间可有多条路径
“结点数 n，边数 $n-1$”	×	图的边数可以多于或少于 $n-1$

同一个小算例：对图 5-9 左边的树（A→B、A→C）与右边的“三角形图”（A–B、B–C、C–A）从 A 出发做层序/BFS。树：A、B、C，三步结束；图：A 先发现 B、C；再处理 B 时又看到 C，处理 C 时又看到 B——**若不标记，C 会被重复入队**；加上标记后结果同为 A,B,C。

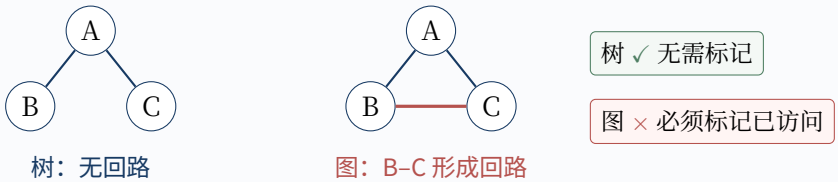


图 5-9　树与带回路的图：红色边 B–C 让遍历有可能“绕回来”，这就是图遍历需要“已访问”标记的原因。

学完这里，你应该能回答

1. 树的递归定义是什么？度、层、高度、叶子、内部结点分别指什么？“高度”有哪两种约定？
2. 二叉树的 5 条性质各是什么？$n_0 = n_2 + 1$ 怎么证？

3. 完全二叉树按 0 编号时，左孩子、右孩子、父结点的编号公式是什么？
4. 四种遍历的访问顺序是什么？层序遍历为什么用队列？
5. 怎样由先序 + 中序还原二叉树？为什么先序 + 后序不一定唯一？
6. 为什么先序序列作入栈次序时，中序序列是出栈序列？
7. “左孩子—右兄弟”规则是什么？表达式并行计算时间为什么是高度−1？

到这里，我们已经解决了什么？下一步还缺什么？

我们有了表示层次关系的二叉树，并掌握了遍历、还原和转换。接下来看一个让树**“有用”**的重要应用：给每个叶子一个权重，让常用的字符走短路径——这就是**哈夫曼树与前缀编码**。后续：二叉树是二叉查找树、堆与平衡树的基础。

第 6 章　哈夫曼树与前缀编码

本章回答 想把一段文字压缩得最短，应该让**常出现的字符用短码、少出现的用长码**。可长短不一的码怎么保证“不会读岔”？哈夫曼树如何一步步造出最优方案？（对应试卷：选择 10，简答 6。）

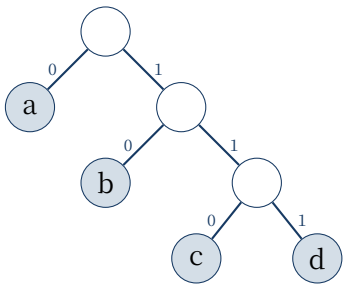
6.1　前缀编码：为什么“不会读岔”

前缀编码

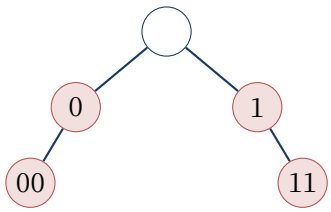
一组编码中，**任何一个码都不是另一个码的前缀**，称为前缀编码（prefix code）。这样接收方从左往右读比特流，**读到某个完整码字就可以立即断开**，不会有歧义。

反例：码集 $\{0, 1, 00, 11\}$ 中 0 是 00 的前缀，收到 00 时无法判断是“0 0”还是“00”。

把编码画成二叉树：左边记 0、右边记 1，每个码字对应从根出发的一条路径。前缀编码⇔所有码字都在叶子上（若码字落在内部结点，它就是后代码字的前缀）。



$\{0, 10, 110, 111\}$：码字全在叶子 ⇒ 前缀码



$\{0, 1, 00, 11\}$：码字在内部结点 ⇒ 不是

图 6-1　左：全部码字都在叶子，是前缀码；右：0、1 是内部结点又是码字，所以 0 是 00 的前缀。

对应试卷：选择 10　[原图识别]

“下述编码中（　）不可能是前缀编码”。逐项检查：A. {00,01,10,11} 等长，互不为前缀 ✓；B. {0,1,00,11} 中 0 是 00 的前缀 ×；C. {0,10,110,111} ✓；D. {1,01,000,001} ✓。**答案 B**。（等长编码一定是前缀码，因为不同的等长串不可能一个是另一个的真前缀。）

6.2 带权路径长度 WPL

带权路径长度

叶子 i 的权为 w_i、从根到它的路径长度（边数）为 l_i，则 WPL $=\sum_i w_i l_i$。若 w_i 是字符出现次数，WPL 就是**整篇文章编码后的总比特数**。哈夫曼树是 WPL 最小的二叉树。

6.3 哈夫曼算法：每次合并最小的两个

构造步骤

(1) 把每个权值看作一棵只有一个结点的树，放入集合；(2) 取出**权最小的两棵**，作为左右子树合成新树，新根的权 = 两者之和，放回集合；(3) 重复直到只剩一棵。n 个叶子共合并 $n-1$ 次，树有 $2n-1$ 个结点，**没有度为 1 的结点**。

对应试卷：简答 6　$W=(5,2,9,11,8,3,7)$　[教学补全]

排序：2,3,5,7,8,9,11。依次合并：

① 2+3=5；② 5+5=10；③ 7+8=15；④ 9+10=19；⑤ 11+15=26；⑥ 19+26=45。

$\mathrm{WPL}=2\cdot4+3\cdot4+5\cdot3+9\cdot2+11\cdot2+7\cdot3+8\cdot3=120$。

快速算法：WPL = 所有**内部结点权之和** $=5+10+15+19+26+45=120$（每个叶子的权在它到根的每条边上都被加了一次）。

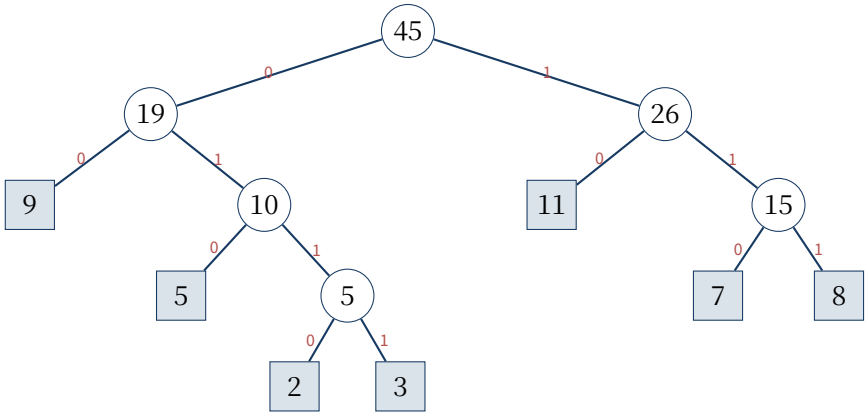
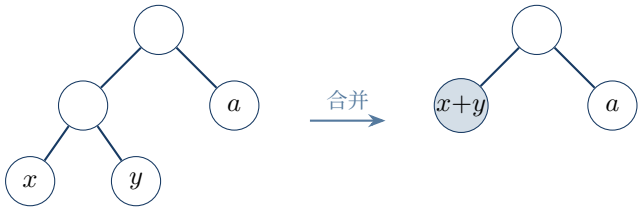


图 6-2　$W=(5,2,9,11,8,3,7)$ 的哈夫曼树（方框为叶子，边上 0/1 为编码）。左小右大只是习惯，左右互换后仍是哈夫曼树，WPL 不变。

6.4 为什么“最小两个合并”是对的（证明思路）

贪心选择的直观论证

设 x,y 是权最小的两个叶子。取任意最优树，若 x,y 不是最深的一对兄弟叶子，把它们与最深的兄弟叶子 a,b 交换位置：x,y 的深度增大、a,b 的深度减小，由于 $w_x\le w_a$、$w_y\le w_b$，交换后 WPL 不增。所以**存在一棵最优树，x,y 是最深层的兄弟**。把 x,y 合并成权为 w_x+w_y 的新叶子后，问题规模减 1，且 WPL 恰好比原来少 w_x+w_y；对 n 归纳即得算法最优。



x, y 最小，已是深层兄弟　　规模减 1，WPL 少 $w_x + w_y$

图 6-3　合并两个最小叶子后得到规模更小的同类问题。

常见错误

(1) 每一步必须在**当前所有树的根**里挑最小两个（包括刚合成的新根），不是只在原始权值里挑；(2) 内部结点权**不能漏加**，也不能把叶子权重复加；(3) 权相同时哈夫曼树**不唯一**，但 WPL 唯一；(4) 路径长度数**边数**，不是结点数。

生活中的它 · 电报码与压缩软件

场景：摩尔斯电码里 E 只用一个点，Q 用“长长点长”，因为 E 常用。**对应**：权 = 字母频率，码长 = 路径长度。**小估算**：4 个字符频率 $0.5, 0.25, 0.125, 0.125$，哈夫曼码长 $1, 2, 3, 3$，平均 1.75 位，比等长 2 位省 12.5%。**边界**：频率完全相同时，哈夫曼码几乎等长，没有压缩收益；实际压缩（如 ZIP 的 Deflate）还要把哈夫曼表一并存下来。

对照：哈夫曼码 ↔ 信息论中的熵

一句话定位：香农熵 $H = -\sum p_i \log_2 p_i$ 给出“平均每符号至少需要多少比特”的下界，哈夫曼码是逐符号编码里最接近它的方案：$H \le \bar{L} < H + 1$。

维度	哈夫曼码（数据结构）	熵（信息论）
对象	一棵具体的编码树	概率分布的理论下界
结果	整数码长	实数（可为小数位）
概率恰为 2^{-k} 时	平均码长 $= H$	H
其他情形	略大于 H（不到 1 位）	不可达

熵中的结论	迁移到哈夫曼	理由或反例
概率越均匀，熵越大	✓	权越接近，哈夫曼树越接近满二叉树，码越长
平均码长不小于熵	✓	Kraft 不等式
平均码长一定等于熵	×	概率 $(0.6, 0.4)$：码长都为 1，$H \approx 0.97$
可逐符号编码达到熵	△	仅当概率是 2 的负整数次幂；一般需分组或算术编码

学完这里，你应该能回答

1. 什么是前缀编码？为什么它等价于“码字都在二叉树的叶子上”？
2. WPL 的含义是什么？为什么等于所有内部结点权之和？
3. 哈夫曼算法的步骤？n 个叶子的哈夫曼树有多少结点、有没有度为 1 的结点？
4. 哈夫曼树是否唯一？WPL 是否唯一？
5. 哈夫曼码的平均长度与熵的关系是什么？

本章之后

至此，从线性表、栈、队列到树与哈夫曼树，我们走完了期中考试的全部知识。下面把它们**串成知识地图**，再用 38 道练习检验。后续课程中的堆、二叉查找树、图与最短路径，都是这里“结点 + 指针 + 递归”思想的延伸。

第 7 章　知识总览与题目对照

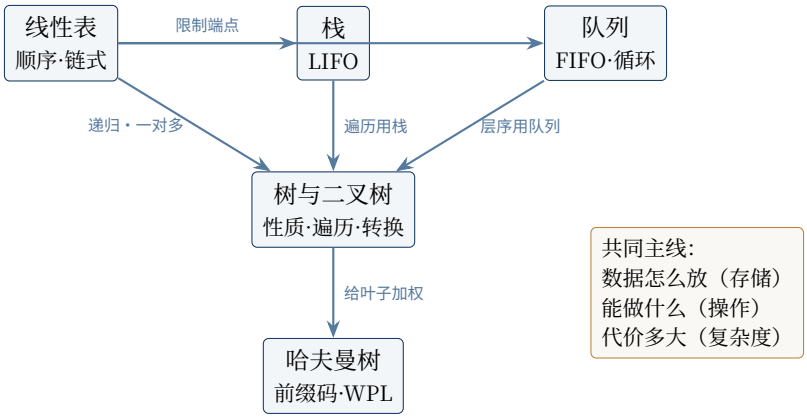


图 8-1　全书知识地图：线性表是起点；栈、队列是它的“受限版本”；树是“一对多”；哈夫曼树是带权的树。

7.1 知识点—原卷题—练习题对照

知识点	原卷题（章）	本讲义练习题号
顺序存储与地址计算	选 6、填 3（第 1 章）	1、2、11、16
链表的插入、删除、逆置	填 2、6、7、简 2、编程（第 2 章）	5、20、25
栈的出栈序列	选 2–4、填 1、5、简 3（第 3 章）	3、12、17、21、30
循环队列公式	选 5、填 1（第 4 章）	4、15、27、38
完全二叉树性质	选 7、填 4（第 5 章）	6、18、28、31、32、33
遍历与还原	选 9、简 5（第 5 章）	7、13、22、26
表达式树	选 8（第 5 章）	29
森林与二叉树转换	简 4（第 5 章）	10、23
前缀编码	选 10（第 6 章）	9
哈夫曼树与 WPL	简 6（第 6 章）	8、14、19、24、34、35
跨学科：信息论、文件系统、信号缓冲	无	36、37、38

教学补全 · 覆盖情况

原卷 10+7+6+2 题 ⇒ 全部知识点均在第 1–6 章中有图解与例子；练习 38 题覆盖上表所有行。题号的“原卷序号”以本讲义第一部分“原卷重新编码”的编号为准。

第 8 章　分层练习（共 38 题）

> **怎么用这一章** 基础题 25 道（题 1–25，对应原卷考法）→ 提高题 10 道（题 26–35）→ 跨学科题 3 道（题 36–38）。题目旁的插图只给“题目本身”的信息，**不含答案**；答案、详解和更丰富的图在后面两章。建议先独立做，再对照。

一、基础题（25 题）

（一）单项选择题（题 1–10）

1. 基础 · 选择　关于线性表的**顺序存储**与**链式存储**，下列叙述正确的是（　）。

A. 顺序存储可以按下标随机访问任意元素
B. 链式存储要求所有结点占用一段连续的内存
C. 顺序表在表头插入元素的时间为 $O(1)$
D. 单链表可以在 $O(1)$ 时间内访问第 i 个结点

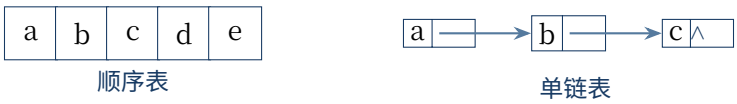


图 E1　一段连续的格子，与一串靠指针串起的结点。

2. 基础 · 选择　数组 `a[0..9]` 每个元素占 4 字节，首地址（`a[0]`）为 1000，则 `a[6]` 的地址是（　）。

A. 1018
B. 1024
C. 1028
D. 1060

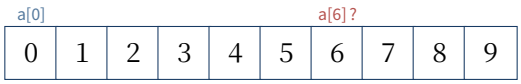


图 E2　10 个元素的数组，下标从 0 开始。

3. 基础 · 选择　入栈序列为 1,2,3,4（每个元素入栈后可随时出栈），下列**可能**的出栈序列是（　）。

A. 3,1,4,2
B. 4,3,1,2
C. 2,4,3,1
D. 1,4,2,3

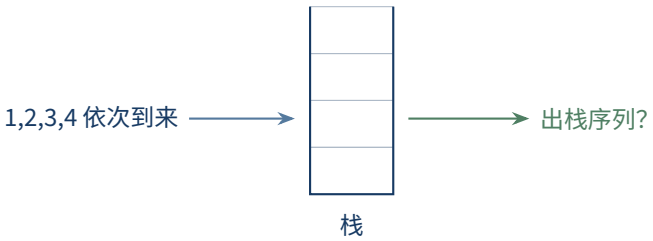


图 E3　元素按 1,2,3,4 的顺序进入同一个栈。

4. 基础 · 选择　循环队列存放在数组 `A[0..7]`（共 8 个单元）中，`front` 指向队头元素，`rear` 指向队尾元素的**下一个**位置。当 `front=5`，`rear=2` 时，队列中的元素个数是（　）。

A. 3
B. 4
C. 5
D. 6

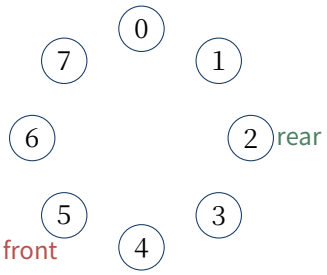


图 E4　8 个单元首尾相接，标出 front 与 rear 的位置。

5. 基础 · 选择　在单链表中，结点 p 之后插入新结点 s，正确的语句顺序是（　）。

A. `p->next = s; s->next = p->next;`
B. `s->next = p->next; p->next = s;`
C. `s->next = p; p->next = s;`
D. `p->next = s->next; s->next = p;`

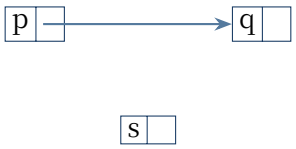


图 E5　p 后面原来是 q；新结点 s 还没接入。

6. 基础 · 选择　一棵有 12 个结点的**完全二叉树**，叶子结点个数为（　）。

A. 4
B. 5
C. 6
D. 7

1	2	3	4	5	6	7	8	9	10	11	12

按层序编号放入数组（编号从 1 开始）

图 E6　12 个结点按层序编号 1–12。

7. 基础 · 选择　某二叉树的先序序列为 ABDECF，中序序列为 DBEAFC，则其后序序列是（　）。

A. DEBFCA
B. DEBCFA
C. DBEFCA
D. EDBFCA

先序	A	B	D	E	C	F
中序	D	B	E	A	F	C
后序	?	?	?	?	?	?

图 E7　已知两种遍历序列，求第三种。

8. 基础 · 选择　权值为 1,2,3,4 的四个叶子构造哈夫曼树，其带权路径长度 WPL 为（　）。

A. 17
B. 19
C. 20
D. 21



图 E8　四个带权叶子，尚未连接。

9. 基础 · 选择　下列编码集合中，可以作为**前缀编码**的是（　）。

A. $\{00, 01, 10, 110\}$
B. $\{0, 1, 10, 11\}$
C. $\{01, 011, 00, 1\}$
D. $\{0, 01, 10, 11\}$

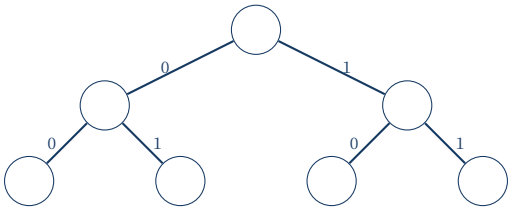


图 E9　读码字的方法：从根出发，向左走记 0，向右走记 1。

10. 基础 · 选择　森林 F 有三棵树，结点数分别为 4、3、2。按“左孩子—右兄弟”规则转成二叉树后，**根结点的右子树**含有的结点数是（　）。

A. 3
B. 4
C. 5
D. 9

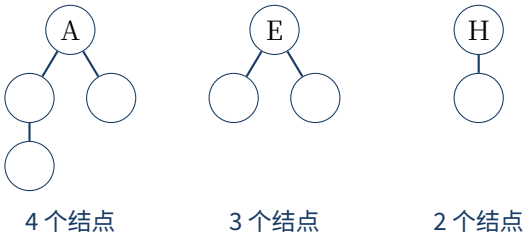


图 E10　森林：三棵树，根分别为 A、E、H。

（二）多项选择题（题 11–15）

11. 基础 · 多选　关于顺序表与链表，下列说法**正确**的有（　）。

A. 顺序表可以按下标 $O(1)$ 访问元素
B. 已知结点位置时，链表的插入删除不必移动其他元素
C. 链表可以在 $O(1)$ 时间随机访问第 i 个结点
D. 顺序表需要预先分配一段连续空间

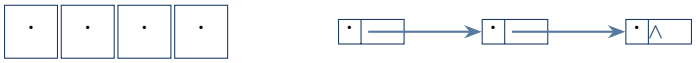


图 E11　左：连续存放；右：靠指针串联。

12. 基础 · 多选　关于**栈**，下列说法正确的有（　）。

A. 栈遵循后进先出（LIFO）

B. 括号匹配检查可以用栈完成
C. 栈只能用数组实现，不能用链表实现
D. 函数的递归调用依赖栈来保存返回现场

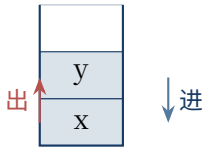


图 E12　栈的进出都在同一端。

13. 基础 · 多选　关于二叉树的遍历，下列说法正确的有（　）。

A. 先序序列与中序序列可以唯一确定一棵二叉树
B. 先序序列与后序序列可以唯一确定一棵二叉树
C. 层序遍历通常借助队列实现
D. 中序序列与后序序列可以唯一确定一棵二叉树

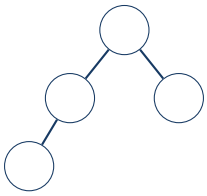


图 E13　一棵二叉树的形状（结点内容省略）。

14. 基础 · 多选　关于哈夫曼树，下列说法正确的有（　）。

A. 哈夫曼树中没有度为 1 的结点
B. n 个叶子的哈夫曼树共有 $2n-1$ 个结点
C. 同一组权值构造出的哈夫曼树形状一定唯一
D. 权值较大的叶子不会比权值较小的叶子离根更远



图 E14　四个叶子权值。

15. 基础 · 多选　设循环队列数组大小为 N，`front` 指队头，`rear` 指队尾下一位，并规定留一个空单元区分空满。下列正确的有（　）。

A. 队空的条件是 `front == rear`
B. 队满的条件是 `(rear+1)%N == front`
C. 元素个数为 `(rear-front+N)%N`
D. 入队后只需 `rear = rear + 1`，无需取模

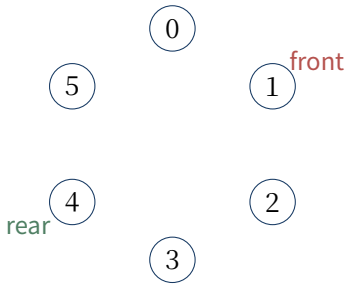


图 E15　6 个单元的环。

（三）填空题（题 16–20）

16. 基础 · 填空　二维数组 a[4][5] 按**行优先**存储，a[0][0] 地址为 100，每个元素占 2 字节，则 a[2][3] 的地址为____。

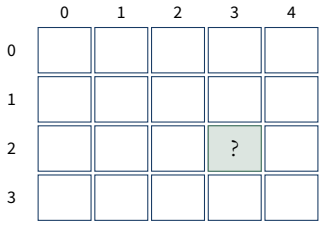


图 E16　4 行 5 列，绿色格为 a[2][3]。

17. 基础 · 填空　入栈序列为 1,2,3,4，得到的出栈序列为 2,1,4,3，则栈的最小容量至少为____。

入栈次序	1	2	3	4
出栈序列	2	1	4	3

图 E17　已知入栈次序与出栈序列。

18. 基础 · 填空　一棵有 100 个结点的完全二叉树，其层数（根为第 1 层）为____，叶子结点数为____。

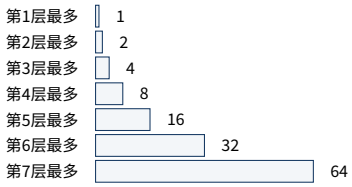


图 E18　各层最多可容纳的结点数。

19. 基础 · 填空　权值为 3,5,6,9 的哈夫曼树的带权路径长度 WPL = ____。



图 E19　四个待合并的叶子。

20. 基础 · 填空　用**头插法**依次把 a、b、c 插入一个带头结点的空单链表，最终从头到尾的顺序是____。

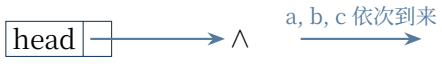


图 E20　开始时链表为空。

（四）解答题（题 21–25）

21. 基础 · 解答　用**栈**判断下面两个括号串是否匹配，写出每读入一个字符后栈的内容：(1) [()]；(2) [(])。



图 E21　两个待检查的括号串。

22. 基础 · 解答　已知二叉树的先序序列为 1 2 4 5 3 6，中序序列为 4 2 5 1 6 3。画出这棵二叉树，并写出后序序列。

先序	1	2	4	5	3	6
中序	4	2	5	1	6	3

图 E22　已知先序与中序。

23. 基础 · 解答　把下面的森林转换成对应的二叉树（画图），并写出二叉树的先序序列。

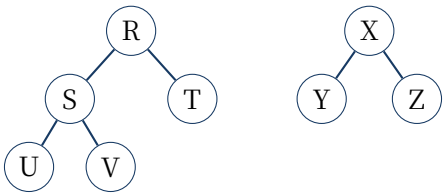


图 E23　由两棵树组成的森林。

24. 基础 · 解答　对权值 $W = (1, 1, 2, 3, 5, 8)$ 构造哈夫曼树，写出每个叶子的编码（左 0 右 1），并计算 WPL。



图 E24　六个叶子权值（斐波那契数）。

25. 基础 · 解答　写出**带头结点单链表就地逆置**的 C++ 函数（不新开结点），并用链表 $1 \to 2 \to 3$ 演示每一步。



图 E25　待逆置的链表（带头结点）。

二、提高题（10 题）

（一）单项选择题（题 26–28）

26. 提高 · 选择　一棵二叉树的先序序列为 ABC，后序序列为 CBA，这样的二叉树共有（　）种。

A. 1
B. 2
C. 3
D. 4

先序	A	B	C
后序	C	B	A

图 E26　先序 ABC，后序 CBA。

27. 提高 · 选择　循环队列数组大小为 10（下标 0–9），初始 `front=rear=0`。依次执行：入队 6 次、出队 4 次、入队 7 次、出队 3 次。此时队列中的元素个数是（　）。

A. 4
B. 5
C. 6
D. 7



图 E27　四次操作依次发生。

28. 提高 · 选择　一棵二叉树中，度为 2 的结点有 15 个，度为 1 的结点有 1 个，则该二叉树的结点总数为（　）。

A. 31
B. 32
C. 33
D. 47

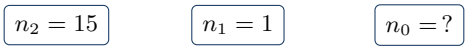


图 E28　按度数分类的结点个数。

（二）多项选择题（题 29–30）

29. 提高 · 多选　表达式 $a + b * c - d/e$ 的表达式树（叶子是操作数、内部结点是运算符）。下列说法正确的有（　）。

A. 后缀表达式为 `abc*+de/-`
B. 前缀表达式为 `-+a*bc/de`
C. 若运算器足够多，最少需要 3 个时间单位完成计算
D. 中序遍历得到的序列无需括号也能保持原式含义

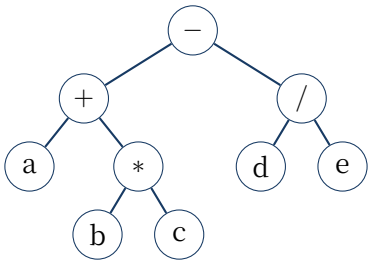


图 E29　$a + b * c - d/e$ 的表达式树。

30. 提高 · 多选　入栈序列为 1,2,3,4,5。下列**合法**的出栈序列有（　）。

A. 3,2,1,5,4
B. 5,4,3,2,1
C. 3,1,2,4,5
D. 2,3,1,5,4



图 E30　五个元素依次进栈。

（三）填空题（题 31–33）

31. 提高 · 填空　一棵含 20 个结点的完全二叉树，结点按层序从 0 到 19 编号。编号为 7 的结点的父结点编号是____，左孩子编号是____，右孩子编号是____。

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19

图 E31　20 个结点的层序编号。

32. 提高 · 填空　用左右指针的二叉链表存储一棵有 12 个结点的二叉树，其中值为 NULL 的指针域共有____个。



图 E32　二叉链表的结点结构。

33. 提高 · 填空　一棵树中度为 3 的结点有 2 个，度为 2 的结点有 1 个，度为 1 的结点有 1 个，则叶子结点有____个。



图 E33　按度分类。

（四）综合题（题 34–35）

34. 提高 · 综合　某文件只含 6 种字符，出现频率（%）为 a:45、b:13、c:12、d:16、e:9、f:5。(1) 构造哈夫曼树并给出编码；(2) 求平均码长；(3) 与定长 3 位编码比较，节省多少比例。

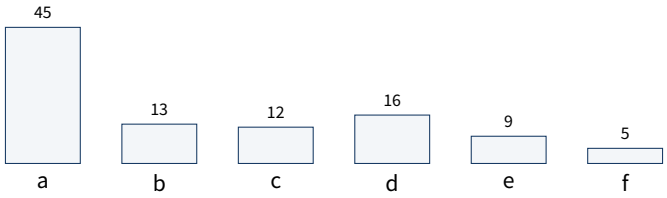


图 E34　各字符出现频率（%）。

35. 提高 · 综合　字符 a,b,c,d,e 的权值分别为 2,4,5,9,10。(1) 构造哈夫曼树，给出各字符编码和 WPL；(2) 与定长 3 位编码的总位数比较；(3) 译码比特串 **0100011011**。



图 E35　五个字符及权值。

三、跨学科题（3 题）

36. 跨学科 · 选择　（信息论）信源有 4 个符号，概率为 $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$。对其构造哈夫曼码，平均码长（位/符号）是（　）。

A. 1.5
B. 1.75
C. 2
D. 2.25

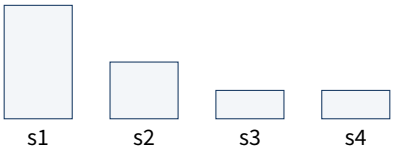


图 E36　四个符号的概率柱状图。

37. 跨学科 · 多选　（操作系统）不考虑硬链接与符号链接，文件系统的目录结构可看作一棵树。下列说法正确的有（　）。

A. 从根目录到某个文件的路径是唯一的
B. 统计一个目录的总大小，应先得到各子目录的大小再汇总
C. 列出目录下所有文件的“递归 `ls`”本质上是一次树的遍历
D. 每个目录最多只能含有两个子项

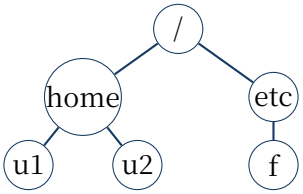


图 E37　一个目录树的片段。

38. 跨学科 · 填空　（信号处理）音频采样缓冲区用环形队列，数组大小 $m = 16$，`front=12`，`rear=4`（`rear` 为下一个写入位置）。缓冲区中待处理的采样点有____个；若采样率为 16 kHz，这些采样对应的时长为____ ms。

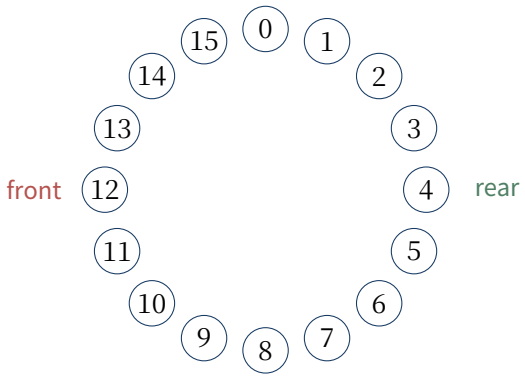


图 E38　16 个单元的环形缓冲区。

第 9 章　答案速查

题号	答案	题号	答案	题号	答案	题号	答案
1	A	11	ABD	21	见详解	31	3；15；16
2	B	12	ABD	22	后序 4 5 2 6 3 1	32	13
3	C	13	ACD	23	先序 R S U V T X Y Z	33	6
4	C	14	ABD	24	WPL=45	34	2.24 位；省 25.3%
5	B	15	ABC	25	见详解	35	WPL=66；省 24 位
6	C	16	126	26	D	36	B
7	A	17	2	27	C	37	ABC
8	B	18	7；50	28	B	38	8；0.5
9	A	19	45	29	ABC		
10	C	20	c, b, a	30	ABD		

教学补全 · 阅读提示

解答题（21、24、25、34、35）的**过程与图**在“详解”一章；题 36–38 另附“四点分析”。

第 10 章　详解与分析

> **阅读方式** 每题格式：**答案** → **思路** → **图** → **选项逐个判断（选择题）** → **易错点**。图中绿色表示正确、红色表示错误或要注意的地方。

基础题详解

题 1　答案：A 顺序表是连续空间，下标直接算出地址。

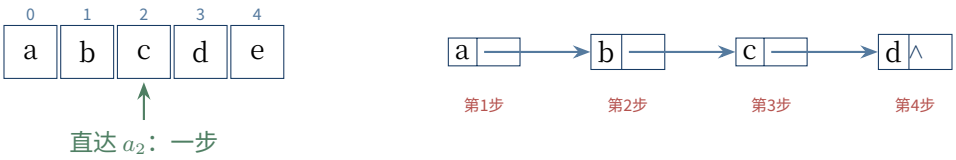


图 A1　数组一步到位，链表只能从头逐个走。

A ✓ 按下标算地址 $O(1)$

B × 链表结点可散落在内存各处

C × 表头插入要后移全部元素 $O(n)$

D × 找第 i 个结点要走 i 步 $O(n)$

易错： 把“插入”的复杂度与“访问”混为一谈：顺序表访问快、插入慢；链表相反。

题 2　答案：B 地址 $= 1000 + 6 \times 4 = 1024$。



图 A2　每格 4 字节，第 6 格的起点是 1000+24。

A × 少算：1000+6×3

B ✓ 1000+6×4=1024

C × 多算一格：下标 7

D × 把 6×10 当成偏移

易错： 地址公式是“首地址 + 下标 × 单元字节数”，**下标从 0 开始**。

题 3　答案：C 用栈模拟：要出的元素若不在栈顶，就必须先把它之前的全部压入；压入后还要保证它在栈顶。

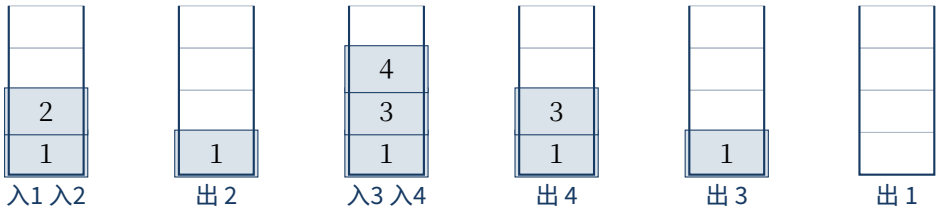


图 A3　选项 C（2,4,3,1）逐步模拟：每一步出栈的都是栈顶，因此合法。

A × 出 3 后栈为 1、2；下一个要 1 但栈顶是 2

B × 出 4、3 后栈为 1、2；要 1 但栈顶是 2

C ✓ 模拟全程可行

D × 出 1、4 后栈内 2、3；要 2 但栈顶是 3

口诀： 若出现“较大数先出，后面较小的两个数顺序是先小后大”（形如 3,1,2），就违反栈序。

题 4　答案：C 个数 $= (\text{rear} - \text{front} + N) \bmod N = (2 - 5 + 8) \bmod 8 = 5$。

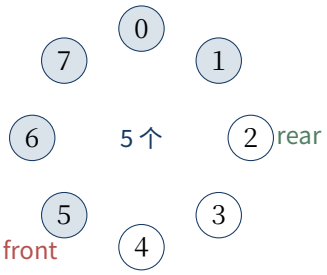


图 A4　蓝色格是从 front 走到 rear 前的格子：5、6、7、0、1，共 5 个。

A × 只数了 rear 之前的 0 与 1 再加一

B × 漏了 7

C ✓ (2−5+8)%8=5

D × 把 rear 所在格也算进去

易错： rear 指向**下一空位**，所以 rear 所在格不属于队列。

题 5　答案：B 先让 s 抓住 q（s->next=p->next），再让 p 指向 s。

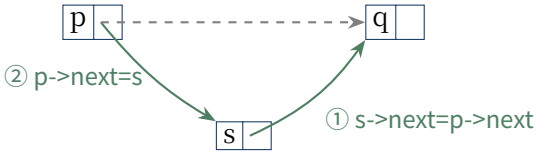


图 A5　先①后②。虚线是原来的链，被②覆盖。



图 A5′　选项 A 的后果：先改 p->next 再读它，读到的是 s 自己。

A × 先改了 p->next 再读它→s->next=s

B ✓ 先保存后继再接入

C × s 指回 p 形成环

D × p->next 被改成 s->next（空值）

口诀： 插入“先接后、再接前”。

题 6　答案：C 完全二叉树 $n = 12$：叶子数 $= n - \lfloor n/2 \rfloor = 6$（编号 7–12 是叶子）。

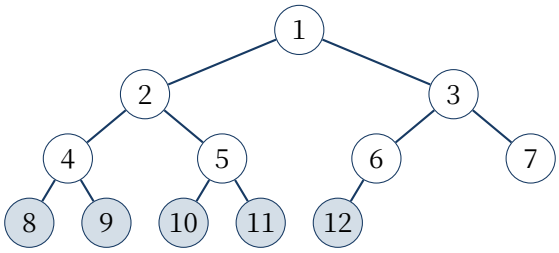


图 A6　12 个结点的完全二叉树。蓝色为第 4 层的叶子 8–12；7 号在第 3 层也没有孩子，所以叶子是 7–12，共 6 个（6 号有左孩子 12，不是叶子）。

A × 只数了最底层的一部分

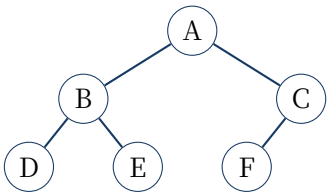
B × 漏数 7 号

C ✓ 6 个：7–12

D × 多数了 6 号（它有孩子 12）

公式： 完全二叉树中 $\lfloor n/2 \rfloor$ 个内部结点（编号从 1），其余 $n - \lfloor n/2 \rfloor$ 个是叶子。

题 7　答案：A 先序首字符 A 是根；中序里 A 左边 DBE 是左子树、右边 FC 是右子树，递归。



后序：D E B F C A

左子树 DEB，右子树 FC，最后根 A

图 A7　还原出的二叉树与后序序列。

A ✓ DEB·FC·A

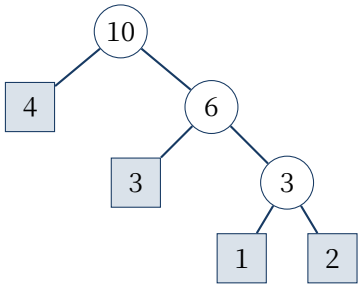
B × C 应在 F 之后

C × B 应在 D、E 之后且 D 在 E 前

D × D 应在 E 前

易错： 后序是“左、右、根”，根永远最后一个。

题 8　答案：B 1+2=3，3+3=6，4+6=10；$\mathrm{WPL} = 3 + 6 + 10 = 19$。



$\mathrm{WPL} = 4 \cdot 1 + 3 \cdot 2 + 1 \cdot 3 + 2 \cdot 3 = 19$

或内部结点和 $3 + 6 + 10 = 19$

图 A8　权值 1,2,3,4 的哈夫曼树。

A × 按链状（1、2、3、4 逐个合）得 20 左右

B ✓ 19

C × 漏加一层内部结点

D × 把 4 放太深

易错： 每次从**全部当前根**里选最小两个，4 不应先于 3 合并。

题 9　答案：A

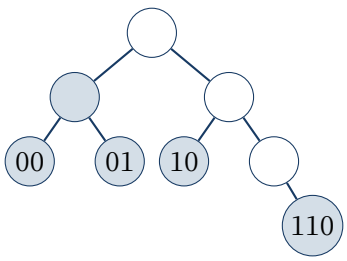


图 A9　选项 A 的码树：00、01、10、110 全部在叶子上（蓝色），所以是前缀码。

A ✓ 四个码都落在叶子

B × 1 是 10、11 的前缀（1 在内部结点）

C × 01 是 011 的前缀

D × 0 是 01 的前缀

判定法： 把码字排序，只需检查**相邻**两项是否前缀关系。

题 10　答案：C 转换后：根 A 的右子树 = 其余各树（E 树 3 个 + H 树 2 个）= 5。

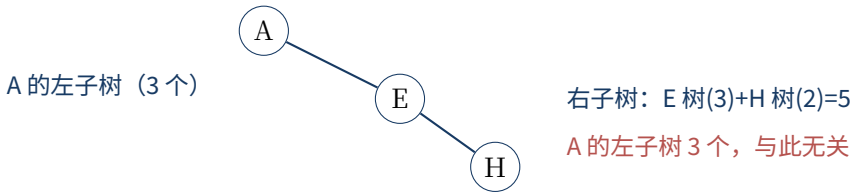


图 A10　各树的根依次用“右指针”相连，后面的树都挂在第一棵树根的右边。

A × 这是 A 的左子树的结点数

B × 误以为包含 A 的结点

C ✓ 3+2=5

D × 把全部 9 个结点都算上

多项选择题（11–15）

题 11　答案：ABD C 错：链表要找第 i 个结点必须从头走，$O(n)$。

A ✓ 下标直达

B ✓ 改指针即可，不移动元素

C × 链表不支持随机访问

D ✓ 数组需要连续空间

题 12　答案：ABD 栈是逻辑结构，可以用数组或链表实现（C 错）。

A ✓ 后进先出

B ✓ 左括号入栈右括号匹配出栈

C × 链栈就是用链表实现

D ✓ 递归的调用栈保存返回地址与局部变量

题 13　答案：ACD 先序+后序不能区分只有一个孩子时“在左还是在右”。

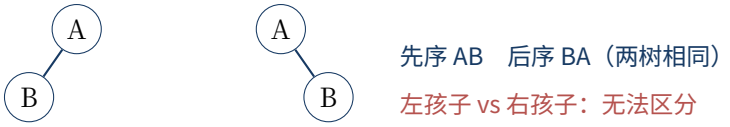


图 A13　先序+后序相同的两棵不同二叉树。

A ✓ 先序找根，中序分左右

B × 反例见图

C ✓ 先进先出的队列

D ✓ 后序末尾是根，中序分左右

题 14　答案：ABD C 错：权相同的结点可以任意先后合并，形状不唯一，WPL 唯一。

A ✓ 要么叶子（度 0）要么两个孩子

B ✓ n 个叶子+(n−1) 个内部结点

C × 形状不唯一

D ✓ 否则交换后 WPL 更小

题 15　答案：ABC D 错：入队要 `rear=(rear+1)%N`，否则越界。

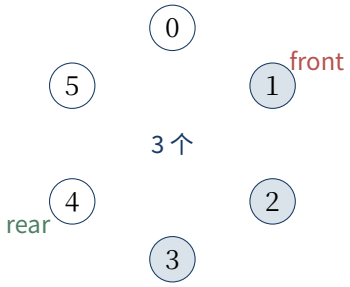


图 A15　N=6，front=1，rear=4：元素个数 $(4-1+6) \bmod 6 = 3$。

A ✓ front==rear 为空	B ✓ 留一个空格区分空满	C ✓ (rear−front+N)%N	D × 必须取模回绕

填空题（16–20）

题 16　答案：126　偏移 $= (2 \times 5 + 3) \times 2 = 26$，地址 $= 100 + 26 = 126$。

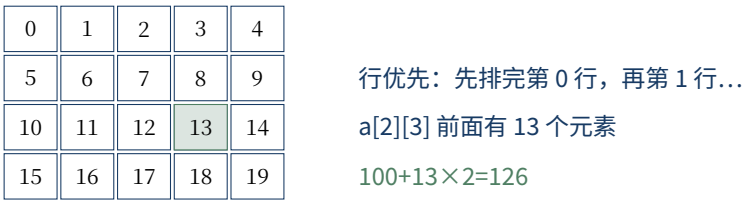


图 A16　格内是行优先的线性序号。

题 17　答案：2　最大栈深：入 1、入 2（深 2）出 2、出 1；入 3、入 4（深 2）出 4、出 3。

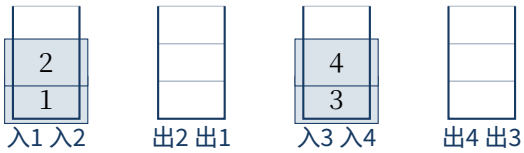


图 A17　栈中元素最多同时有 2 个。

题 18　答案：7；50　$2^6 = 64 \le 100 < 128 = 2^7$，故 7 层；叶子数 $= 100 - \lfloor 100/2 \rfloor = 50$。

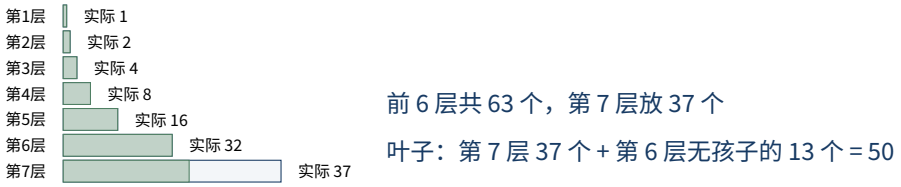


图 A18　100 = 63 + 37。第 6 层 32 个结点中，有 $\lceil 37/2 \rceil = 19$ 个有孩子，其余 13 个是叶子。

题 19　答案：45　3+5=8，6+8=14，9+14=23；$\mathrm{WPL} = 8 + 14 + 23 = 45$。

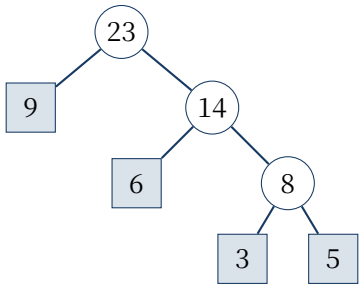


图 A19　内部结点权之和 $8 + 14 + 23 = 45$。

题 20　答案：c, b, a　头插法每次插在头结点之后，后插入的在前。

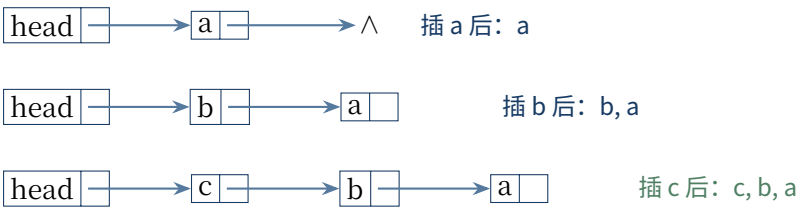


图 A20　头插法三步；尾插法才保持原来的先后顺序。

解答题（21–25）

题 21　答案： (1) 匹配；(2) 不匹配。 规则：左括号入栈；右括号要求与栈顶是一对，匹配则出栈；结束时栈空才算匹配。

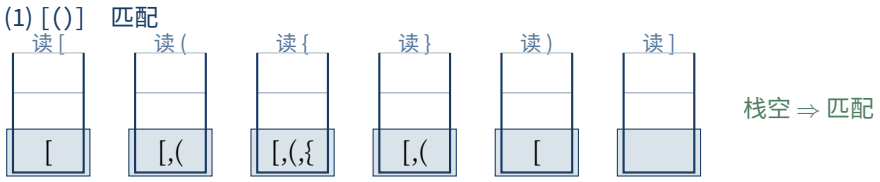


图 A21-1　(1) 逐字符的栈：右括号总能和栈顶配对。

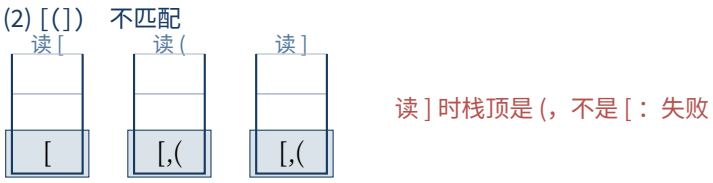


图 A21-2　(2) 在读到] 时失败。

易错： 只数左右括号个数相等并不够，必须**类型和嵌套顺序**都对。

题 22　答案： 后序 **4 5 2 6 3 1**。 先序首 1 是根；中序中 1 左边 425 是左子树，右边 63 是右子树。

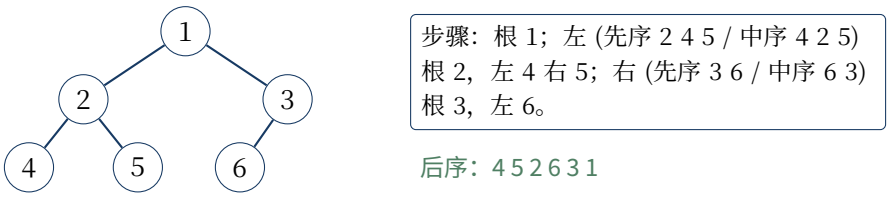


图 A22　还原的二叉树。

题 23　答案： 先序 **R S U V T X Y Z**。 规则：每个结点“左孩子 = 第一个孩子，右孩子 = 下一个兄弟”，各棵树的根互为兄弟。

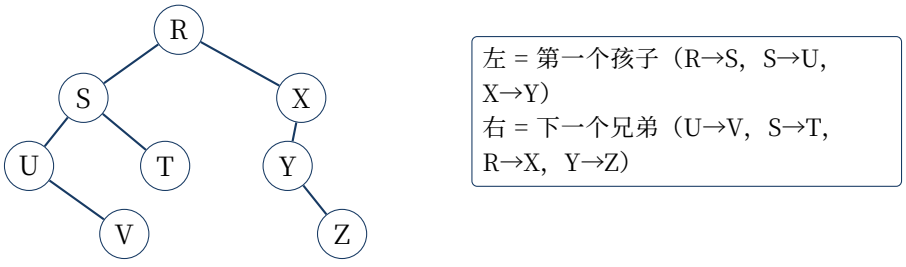
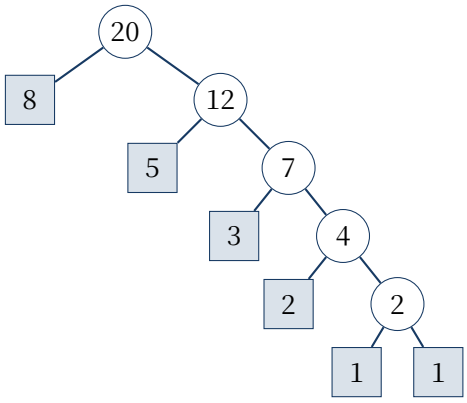


图 A23　转换结果；该二叉树的先序序列恰为 R S U V T X Y Z，与森林先序一致。

题 24　答案： WPL = **45**。 合并顺序：1+1=2；2+2=4；3+4=7；5+7=12；8+12=20。



码字（左 0 右 1）：
8：0　5：10　3：110
2：1110　1：11110　1：11111

WPL $= 8+10+9+8+5+5 = 45$
或 $2+4+7+12+20 = 45$

图 A24　斐波那契权值给出“最不平衡”的哈夫曼树（一条长长的右链）。

易错： 第二步合并的是原来的 2 与新得的 2（**含新结点**一起排序），而不是 2 与 3。

题 25　答案： 见下面代码。思路：把结点逐个摘下，用**头插法**接到头结点后面。

```
void Reverse(Node *h) {            // h 为头结点
    Node *p = h->next, *q;
    h->next = NULL;                // 先把链表断开
    while (p) {
        q = p->next;               // 保存后继
        p->next = h->next;         // 头插
        h->next = p;
        p = q;                     // 取下一个
    }
}
```

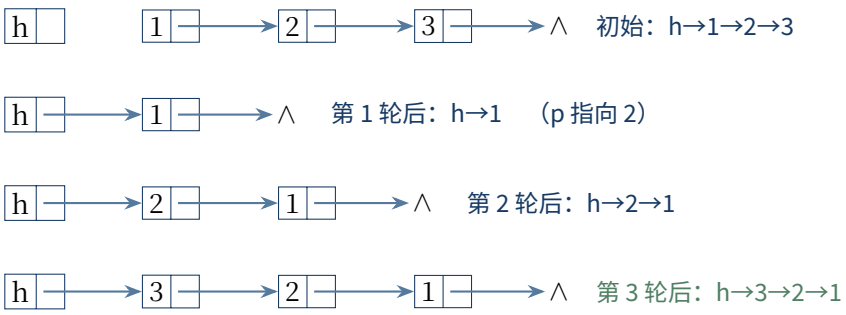


图 A25　每轮把一个结点摘下，插到头结点之后。时间 $O(n)$，额外空间 $O(1)$。

易错： `q=p->next` 必须先保存，否则改了 `p->next` 后链表就断了。

提高题详解

题 26　答案：D 先序 ABC 且后序 CBA：A 是根，B 是 A 的**唯**一孩子，C 是 B 的唯一孩子；每个唯一孩子都可以放左或放右，$2 \times 2 = 4$。



图 A26　四种形状，先序都是 ABC、后序都是 CBA。

A × 只考虑了一种链 | B × 只区分了一次选择 | C × 漏掉一种 | D ✓ 2×2=4

题 27　答案：C 个数 $= 6-4+7-3 = 6$；front$= (0+4+3) \bmod 10 = 7$，rear$= (6+7) \bmod 10 = 3$，验证 $(3 - 7 + 10) = 6$。

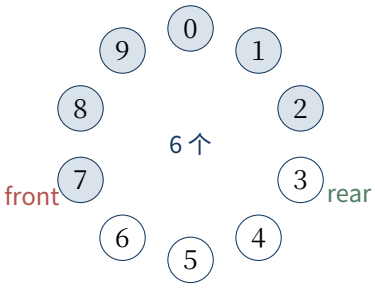
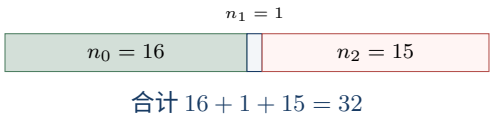


图 A27　front=7，rear=3，蓝色格 7、8、9、0、1、2 共 6 个。

A × 只算了第一轮净增 | B × 少加一次入队 | C ✓ 6−4+7−3=6 | D × 多算 rear 所在格

题 28　答案：B $n_0 = n_2 + 1 = 16$，$n = n_0 + n_1 + n_2 = 16 + 1 + 15 = 32$。

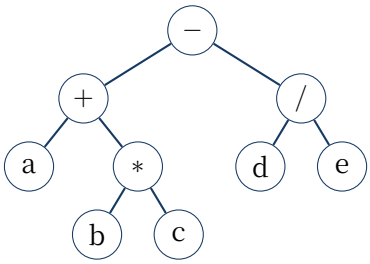


合计 $16 + 1 + 15 = 32$

图 A28　叶子比二度结点多 1。

A × 漏了 n1 | B ✓ 16+1+15=32 | C × n0 取了 17 | D × 把 n0 误算成 31

题 29　答案：ABC



第 3 步：−
第 2 步：+（与 / 同时）
第 1 步：∗ 与 / 同时

后缀：a b c * + d e / -
前缀：- + a * b c / d e

图 A29　层数 4（根为第 1 层），运算时间 $= 4 - 1 = 3$。

A ✓ 左右根的后序 | B ✓ 根左右的先序 | C ✓ 高度−1=3 | D × 中序会丢括号：a+b*c-d

易错： 同一棵树中，∗ 与 / 位于不同分支，可并行；因此“3 步”而非 4 步。

题 30　答案：ABD

C：3,1,2,4,5 不合法

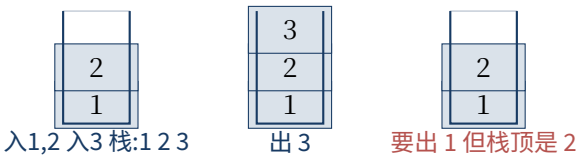
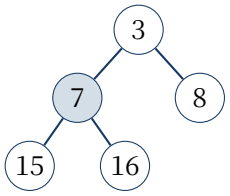


图 A30　选项 C 的失败点：3 出栈后栈内是 1、2，栈顶是 2。

A ✓ 入1 2 3 出 3 2 1；入 4 5 出 5 4	B ✓ 全部入栈再全部弹出	C × 3 之后 1 不在栈顶	D ✓ 入1 2 出 2；入 3 出 3；出 1；入 4 5 出 5 4

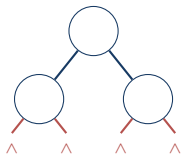
题 31　答案：3；15；16 按 0 编号：父 $= \lfloor (i-1)/2 \rfloor = 3$，左 $2i+1 = 15$，右 $2i+2 = 16$（都 < 20，存在）。



父：$\lfloor (7-1)/2 \rfloor = 3$
左：$2 \cdot 7 + 1 = 15$
右：$2 \cdot 7 + 2 = 16$

图 A31　7 号结点的父亲与孩子。

题 32　答案：13 共 $2n = 24$ 个指针域，其中 $n-1 = 11$ 个被孩子占用，空的 $= 2n - (n-1) = n+1 = 13$。



3 个结点：指针域 6 个，孩子 2 个
空指针 $= 6 - 2 = 4 = 3 + 1$

图 A32　小例子验证 $n+1$ 规律。

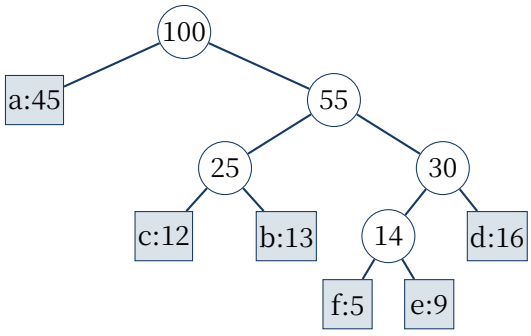
题 33　答案：6 结点总数 $= 1 + \sum(\text{度} \times \text{个数}) = 1 + 3 \cdot 2 + 2 \cdot 1 + 1 \cdot 1 = 10$；$n_0 = 10 - 2 - 1 - 1 = 6$。



总数 10；总分支数 $= 9 = n_1 + 2n_2 + 3n_3$

图 A33　结点数 $=$ 分支数 $+1$。

题 34　答案： 平均码长 **2.24 位**，比定长 3 位省 **25.3%**。 合并：5+9=14；12+13=25；14+16=30；25+30=55；45+55=100。



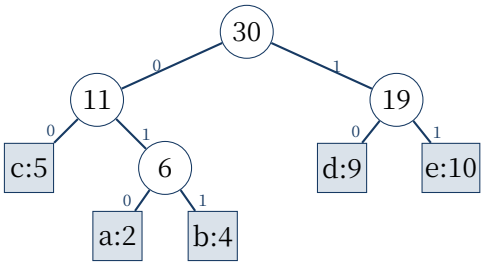
a:0　c:100　b:101
d:111　f:1100　e:1101

平均码长
$0.45 \cdot 1 + 0.12 \cdot 3 + 0.13 \cdot 3$
$+0.16 \cdot 3 + 0.05 \cdot 4 + 0.09 \cdot 4 = 2.24$

图 A34　哈夫曼树与编码。定长 3 位需要 3 位/字符，节省 $(3 - 2.24)/3 \approx 25.3\%$。

补充： 熵 $H \approx 2.22 \le 2.24$，哈夫曼码已非常接近理论下界。

题 35　答案： (1) **WPL=66**；(2) 定长需要 $3 \times 30 = 90$ 位，省 24 位；(3) 译码 **a c e b**。



a:010　b:011　c:00
d:10　e:11

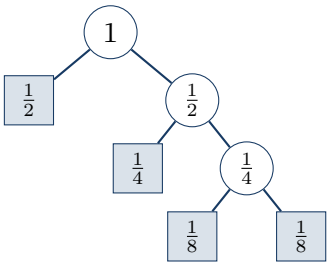
WPL $= 2\cdot3+4\cdot3+5\cdot2+9\cdot2+10\cdot2=66$
定长：$3\times30=90$

译码：010 | 00 | 11 | 011 ⇒ a c e b

图 A35　哈夫曼树、码表与译码切分：前缀码保证“读到完整码字就能切开”。

跨学科题详解

题 36　答案：B　合并 $\frac18+\frac18=\frac14$；$\frac14+\frac14=\frac12$；$\frac12+\frac12=1$。码长 1、2、3、3，平均 $0.5\cdot1+0.25\cdot2+0.125\cdot3\cdot2=1.75$。



码：0　10　110　111

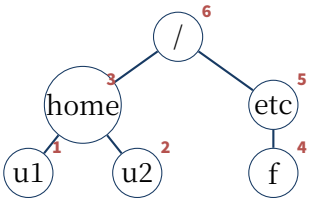
平均码长 $=1.75$ 位
熵 $H=\sum p\log_2\frac1p=0.5+0.5+0.375+0.375=1.75$

图 A36　概率恰为 2^{-k} 时，哈夫曼码与熵完全相等。

A × 比熵还小是不可能的　B ✓ 1.75　C × 等长 2 位　D × 把最长码长 3 当平均

四点分析　**知识点：**哈夫曼树构造、WPL（平均码长）。**跨学科联系：**信息论中的熵是平均码长的下界。**思维方式：**把“频率”看成“权”，把“编码”看成“路径”。**边界：**概率不是 2^{-k} 时哈夫曼码略高于熵；全等概率时码长几乎相同。

题 37　答案：ABC



红色数字：后序汇总顺序
u1,u2 → home → etc/f → /

图 A37　先算子目录大小，最后汇总到根：这就是后序遍历（du 的做法）。

A ✓ 树中从根到结点路径唯一　B ✓ 后序：先子后父　C ✓ 递归 ls 是先序遍历　D × 一般树结点孩子数不限

四点分析　**知识点：**一般树与遍历。**跨学科联系：**操作系统的目录管理。**思维方式：**把层级结构当作树，再按需要选遍历顺序。**边界：**硬链接、符号链接会让目录成为图而不是树。

题 38　答案：8；0.5　个数 $=(4-12+16)\bmod16=8$；时长 $=8/16000\,\text{s}=0.5\,\text{ms}$。

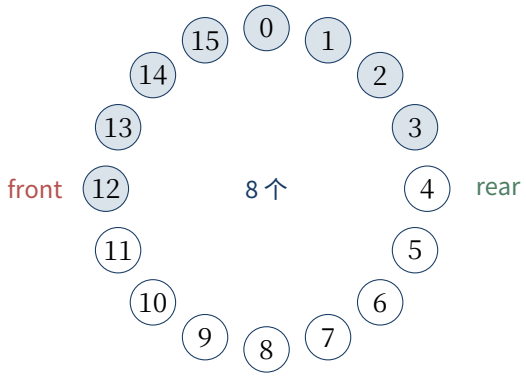


图 A38　蓝色格 12–15 和 0–3 共 8 个采样点。

四点分析　**知识点：**循环队列个数公式 $(r-f+m) \bmod m$。**跨学科联系：**音频采样的环形缓冲区，把“单位”换成时间。**思维方式：**先算个数，再按速率换算单位。**边界：**留一个空格区分空满时，最多存 $m-1=15$ 个采样；写入快过读取就会丢数据。

第 11 章　附录：总结与参考

11.1　错误与易错总结

位置	易错内容	正确做法
原卷填空 5	同学答案写成 5	栈最大深度应为 4（见“答案独立核验与原图纠错”）
原卷填空 7	循环条件写错	`q != first`，末尾 `first->link = p;`
原卷填空 6	删除后继缺 `delete` 说明	先保存再释放，防内存泄漏
原卷编程 1	漏写 `p=h`	从头结点开始遍历
插入 / 删除链表	先改指针丢失后继	先保存后继，再改指针
循环队列	个数漏加 N	$(rear-front+N) \bmod N$
前缀码	只比较相邻两个以外的码	排序后检查相邻码
哈夫曼	合并时漏掉新结点	每次在全部根中选最小两个
先序 + 后序	以为唯一	一个孩子时无法分左右

11.2　解题步骤总结

题型	步骤
出栈序列合法性	逐个模拟：要出的元素不在栈顶则继续压入；压完仍不在栈顶则不合法
由遍历序列还原	先序/后序找根 $\rightarrow$ 中序切左右 $\rightarrow$ 递归
哈夫曼树	排序 $\rightarrow$ 合并最小两个 $\rightarrow$ 放回 $\rightarrow$ 重复；WPL = 内部结点权之和
循环队列	空：`front==rear`；满：`(rear+1)%N==front`；个数：`(rear-front+N)%N`
完全二叉树	0 编号：左 $2i+1$，右 $2i+2$，父 $\lfloor (i-1)/2 \rfloor$；叶子数 $n-\lfloor n/2 \rfloor$

11.3 相似概念对照汇总

概念对	共同点	区别
顺序表 / 链表	都是线性表	随机访问 vs 灵活插入删除
栈 / 队列	都限制端点	后进先出 vs 先进先出
完全二叉树 / 满二叉树	结构整齐	满二叉树每层都满
先序 / 中序 / 后序	都访问每个结点一次	根的位置不同
树 / 二叉树	都是层次结构	二叉树分左右且度 ≤ 2
哈夫曼树 / 二叉查找树	都是二叉树	前者按权重最小路径，后者按键值有序

11.4 跨学科联系汇总

数据结构	其他领域中的对应
栈	函数调用栈（程序设计）、撤销（编辑器）、括号检查（编译器）
循环队列	音频缓冲、网络收发缓冲（信号处理、通信）
树	文件目录（操作系统）、语法树（编译原理）、表达式求值
哈夫曼树	数据压缩、信息熵（信息论）
链表	内存空闲块管理（操作系统）、播放列表

11.5 后续课程联系

二叉查找树、平衡树、堆与优先队列（《数据结构》后半部分）；图与最短路径、拓扑排序；操作系统的进程调度与内存管理；编译原理的语法树与栈；信息论与压缩算法。

11.6 参考资料

原始材料：某高校　2025—2026 学年第 1 学期《数据结构与算法》期中试卷（A）及同学手写答案（扫描件，已核验）。

教材通用知识：栈、队列、树与哈夫曼树的标准结论（各数据结构教材通用）。

文献：Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. Proceedings of the IRE, 40(9), 1098–1101.

在线条目（用于核对历史与定义）：Wikipedia: Stack (abstract data type)、Linked list、Huffman coding。

说明：页面中的图均为本讲义自行绘制，题目为依据原卷考法改编或原创。